NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 14: Comprehensive Course Review

Self-Study and Practice | Weeks 1–13

Yanwei Wang
School of Engineering
Nazarbayev University

ENG 101 – 2L | 6 ECTS | Fall 2026

Predict. Trace. Explain. Test. Interpret.

A topic-organized reading companion to the revised 240-page review, with commented code, worked practice, and an integrated engineering report.

About these notes

These notes accompany *Programming for Engineers: Comprehensive Course Review—Self-Study and Practice | Weeks 1–13*, the revised 240-page Week 14 slide deck by Yanwei Wang. That topic-organized revision, rather than the earlier 87-page three-lecture review, is the controlling source. The source is identified by its title, page count, and file checksum in the package's source manifest.

Week 14 introduces no new assessed topic. Its purpose is to reconnect ideas that were first learned separately: a string must be interpreted before it becomes a numerical input; a function's returned value must match its contract; a class must preserve its promised state; an efficient algorithm must still solve the same problem; and a graph must preserve the meaning of the data it displays.

The chapters retain the source's sixteen modules. Modules 1–13 follow the actual course sequence, Module 14 develops the integrated temperature report, Module 15 contains mixed-topic practice, and Module 16 supports targeted revision. Source-page references beside each section allow students and the instructor to compare the notes with the slides. The appendix accounts for all 240 source pages.

Scope and assessment

These materials are for independent study and formative practice. They do not announce examination questions, an examination format, a grading threshold, a deadline, or permitted-resource rules. Consult the current Moodle instructions for assessed work. No three-lecture pacing plan, timed activity, or partner requirement is imposed here.

What has been expanded

The compressed explanations in the slides are developed into connected prose, worked calculations, trace tables, and explicit descriptions of assumptions. All 48 original question identifiers are retained. Solutions remain visible to students and explain why an answer follows. Added code drivers, concrete inputs for source patterns, and repairs are labelled as worked completions rather than attributed to an unseen source implementation.

Every executable code listing includes explanatory comments. A comment explains intent or behavior; it is not an instruction executed by Python. The full package supplies the actual files used for the listings, expected outputs for deterministic demonstrations, tests, and editable figure sources. Incomplete patterns and function excerpts are identified as such. The original review companion modules are preserved unchanged in the reference folder.

Engineering check

Illustrative temperatures, resistances, design categories, and material properties are teaching data, not certified engineering specifications. A result that satisfies a stated computational rule does not establish that a physical system is safe. Keep implementation checks, mathematical assumptions, and real-world validity separate.

How to use the review

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 1–6.

Review in course order, or begin with a question in a topic you find difficult. After an independent attempt, read the explanation and run a changed example. Neither route has a fixed timetable.

1. **Predict.** Write the output, returned value, error, or changed state before running a sample.
2. **Trace.** Identify assignments, branches, loop visits, and return points. Draw references for mutable objects.
3. **Compare.** Locate the first difference between your reasoning and the worked solution.
4. **Run and vary.** Execute the example in a clean process. Change one input and predict again.
5. **Explain and test.** State the contract, units, stopping condition, and independent evidence. Retain a test for a repaired misconception.

Questions and solutions

Identifiers such as `M05-Q2`, `INT-Q1`, and `X08` match the revised slides. Links connect each question and its worked solution. The questions-only workbook supports a first attempt without visible solutions; it does not define an assessment.

Independent code examples

Run examples separately unless an import states a dependency. Reused names do not mean two examples share one model. In particular, the simple `IntroSensor` and checked `Sensor` have different assumptions and interfaces. Do not combine their definitions accidentally or rely on variables left in an earlier notebook cell.

Running the resources

Numbered Python examples use only the standard library. Run from the extracted project root:

```
# Run one independent example from the extracted project.
python examples/q_m05_2.py
```

C++ and Java files require their respective toolchains. MATLAB files are commented reading and native-test exercises; MATLAB execution is not claimed. These tools are not prerequisites for reading the Python review.

After a correct answer, change one input and explain the result again.

Contents

About these notes ii

How to use the review iii

1. Computation, variables, and types 1
1.1. A result is part of an engineering argument . . . 1
1.2. Arithmetic depends on operands as well as operators . . . 1
1.3. Assignment happens when the statement executes . . . 2
1.4. Input, output, and intended meaning . . . 3
1.5. A complete stress calculation with units . . . 3
1.6. From power to energy: the same unit discipline . . . 4
1.7. Practice: attempt before consulting the solutions . . . 4
1.8. Worked solutions and reasoning . . . 5
1.9. Module checkpoint . . . 5

2. Decisions and repetition 6
2.1. One classification or several independent checks? . . . 6
2.2. Boolean expressions should express the actual rule . . . 7
2.3. A range gives candidates, not a guarantee of visits . . . 7
2.4. Initialization and update define the state history . . . 8
2.5. A while loop requires progress and a stopping explanation . . . 8
2.6. A full time step can cross a target . . . 9
2.7. Practice: attempt before consulting the solutions . . . 9
2.8. Worked solutions and reasoning . . . 10
2.9. Module checkpoint . . . 11

3. Strings and numerical search 12
3.1. An index selects a character; a slice selects a sequence . . . 12
3.2. Transform text without silently changing its meaning . . . 12
3.3. Parsing is an agreement about a record format . . . 13
3.4. Guess-and-check is a procedure, not intuition . . . 14
3.5. Bisection retains a root-containing interval . . . 14
3.6. Residual tolerance and input uncertainty differ . . . 15
3.7. What a bracket argument cannot establish . . . 16
3.8. Practice: attempt before consulting the solutions . . . 16
3.9. Worked solutions and reasoning . . . 17

4. Functions and program design 18
4.1. Define the tool, then invoke it . . . 18
4.2. Parameters, arguments, and defaults . . . 18
4.3. Returning, printing, and mutating are different actions . . . 19
4.4. Local names and shared objects . . . 19
4.5. Decomposition and abstraction support each other . . . 20
4.6. A return inside a loop changes the algorithm . . . 21
4.7. Practice: attempt before consulting the solutions . . . 21
4.8. Worked solutions and reasoning . . . 22

5. Lists, Tuples, and Data Integrity **24**
5.1. Choose a representation before processing it . . . 24
5.2. Indices, lengths, and data windows . . . 24
5.3. Track mutation and return values independently . . . 25
5.4. Aliasing, shallow copying, and nested data . . . 25
5.5. Filter without deleting from the collection being traversed . . . 26
5.6. Tuples keep related fields together . . . 26
5.7. Summaries do not replace the measurements . . . 27
5.8. Practice: attempt before consulting the solutions . . . 27
5.9. Worked solutions and reasoning . . . 28
5.10. Collection checkpoint . . . 29

6. Recursion and Dictionaries **30**
6.1. Three obligations of a recursive function . . . 30
6.2. Factorial: finish the return path . . . 30
6.3. Fibonacci: two base cases and overlapping work . . . 31
6.4. Recursion on strings . . . 32
6.5. A dictionary represents associations, not numerical positions . . . 32
6.6. Nested lookup must be checked at the intended level . . . 33
6.7. Practice: attempt before consulting the solutions . . . 33
6.8. Worked solutions and reasoning . . . 34
6.9. Recursion and dictionary checkpoint . . . 35

7. Testing, Debugging, and Explicit Failure **36**
7.1. Correct with respect to a specification . . . 36
7.2. Choose tests for several reasons . . . 36
7.3. Combine black-box and glass-box reasoning . . . 37
7.4. Classify the symptom and read the traceback . . . 37
7.5. Repair one cause and keep the evidence . . . 37
7.6. Exceptions make failure a separate path . . . 37
7.7. Assertions are checks, not permanent input protection . . . 38
7.8. Check the state as well as the result . . . 39
7.9. Practice: attempt before consulting the solutions . . . 39
7.10. Worked solutions and reasoning . . . 40
7.11. Reliability checkpoint . . . 41

8. Classes, Instances, and Methods **42**
8.1. Organize one meaningful entity . . . 42
8.2. Initialization establishes instance data . . . 42
8.3. The dot chooses an object; parentheses perform the call . . . 43
8.4. Equal data does not mean shared identity . . . 44
8.5. Practice: attempt before consulting the solutions . . . 44
8.6. Worked solutions and reasoning . . . 45
8.7. Object checkpoint . . . 45

9. Interfaces, Shared State, and Inheritance **46**
9.1. Validate before committing a state change . . . 46
9.2. Hide representation, not units and responsibilities . . . 47
9.3. Class defaults and instance overrides . . . 48
9.4. A mutable class attribute can share more than intended . . . 48
9.5. Specialize a class without breaking its contract . . . 49
9.6. Practice: attempt before consulting the solutions . . . 50

9.7. Worked solutions and reasoning . . . 51
9.8. Interface checkpoint . . . 51

10. Efficiency: Counts and Orders of Growth 52
10.1. A complete claim begins with the task . . . 52
10.2. Count every relevant operation, not only interesting events . . . 52
10.3. Big-O is an eventual upper bound . . . 53
10.4. Dominant terms and finite-size decisions . . . 53
10.5. Best, worst, and assumed average cases . . . 53
10.6. Recognize the structure behind each growth class . . . 54
10.7. Measure a precisely defined region . . . 54
10.8. Practice: attempt before consulting the solutions . . . 55
10.9. Worked solutions and reasoning . . . 56
10.10. Efficiency checkpoint . . . 56

11. Hidden Work, Recursion, and Memory 57
11.1. A short expression can hide a full scan . . . 57
11.2. Repeated construction can repeatedly copy the prefix . . . 57
11.3. Preprocessing is part of the cost . . . 58
11.4. Recursive cost is local work times the number of calls . . . 58
11.5. A cache changes which subproblems are new . . . 59
11.6. Moving-window means: preserve every requested output . . . 60
11.7. Algebraic equivalence is not identical floating-point behavior . . . 61
11.8. Move unchanged work outside a loop only when valid . . . 61
11.9. Practice: attempt before consulting the solutions . . . 62
11.10. Worked solutions and reasoning . . . 62
11.11. Hidden-work checkpoint . . . 63

12. Searching and Sorting 64
12.1. The search question determines the algorithm . . . 64
12.2. Linear search preserves a checked-prefix invariant . . . 64
12.3. Brute force is systematic candidate enumeration . . . 65
12.4. Bisection search depends on sortedness . . . 65
12.5. Trace both the values and the bounds . . . 66
12.6. Insertion positions are not evidence of membership . . . 66
12.7. Sorting has a contract beyond increasing numbers . . . 67
12.8. Selection sort exposes repeated minimum searches . . . 67
12.9. Merge sort combines already sorted halves . . . 68
12.10. Choose for the complete workload . . . 69
12.11. Practice: attempt before consulting the solutions . . . 69
12.12. Worked solutions and reasoning . . . 70
12.13. Search-and-sort checkpoint . . . 71

13. Programming Beyond Python 72
13.1. Keep the problem and evidence fixed . . . 72
13.2. Distinguish the layers of a programming environment . . . 72
13.3. Integer division can change the answer before assignment . . . 72
13.4. Indexing, shape, and array operations . . . 73
13.5. Read blocks and interfaces from their purpose outward . . . 74
13.6. Equality, aliases, and copies need language-specific attention . . . 74
13.7. Translate a cooling recurrence, not just its characters . . . 75
13.8. Use assistance without transferring responsibility . . . 76

13.9. Practice: attempt before consulting the solutions . . . 76
13.10. Worked solutions and reasoning . . . 77
13.11. Language-transfer checkpoint . . . 78

14. Integrated Engineering Review **79**
14.1. State the complete report contract . . . 79
14.2. Preserve the seven raw records . . . 79
14.3. Decompose the pipeline into visible responsibilities . . . 80
14.4. Parse deliberately and reject unsupported inputs . . . 80
14.5. Keep accepted and rejected paths separate . . . 81
14.6. Interpret the mean alongside the individual observations . . . 82
14.7. Rank complete accepted records . . . 83
14.8. Test the complete route through the program . . . 84
14.9. Explain efficiency without losing string-processing cost . . . 84
14.10. Practice: attempt before consulting the solutions . . . 84
14.11. Worked solutions and reasoning . . . 85

15. Mixed-Topic Practice **86**
15.1. Practice: attempt before consulting the solutions . . . 86
15.2. Worked solutions and reasoning . . . 88

16. Final Checks and Targeted Revision **91**
16.1. Judge readiness by what you can explain . . . 91
16.2. Use an error log to direct the next attempt . . . 91
16.3. Source basis and explicit distinctions . . . 92
16.4. A compact final reference . . . 92

A. Source Alignment and Question Index **93**
A.1. Controlling source and use of earlier materials . . . 93
A.2. Topic map . . . 93
A.3. Question and worked-answer index . . . 94
A.4. Complete slide-page coverage . . . 95

1 Computation, variables, and types

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 7–18.

After reviewing this module

Distinguish a model from an algorithm and a program; trace assignment; choose types by meaning; evaluate Python arithmetic correctly; and derive the units of an engineering calculation before interpreting its output.

1.1. A result is part of an engineering argument

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 8, 13.

An engineering question comes first: what must be calculated, compared, or decided? A model then represents the relevant situation under assumptions. For a uniformly loaded bar, the model may be $\sigma = F/A$. An algorithm is the sequence of operations used to obtain and check a result. A program expresses those operations in a language. The output is evidence produced by that implementation, not a substitute for the original question or the model assumptions.

This distinction is useful during debugging. A program can implement F/A perfectly while using an area in the wrong unit. It can also implement the formula with correct units while the assumption of uniform axial stress is unsuitable. The first problem is an implementation or data-meaning issue; the second concerns the relationship between the model and the actual system. Passing one kind of check does not automatically settle the other.

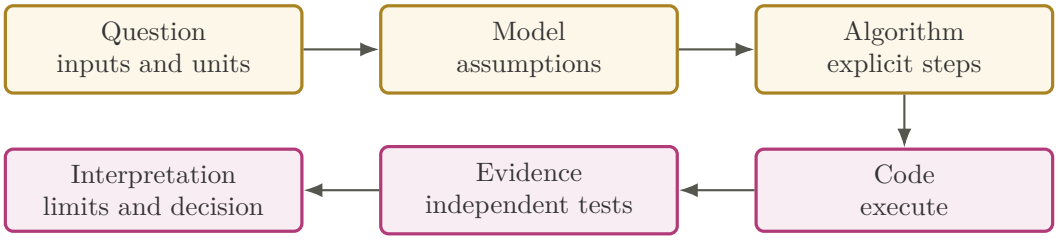


Figure 1.1.: The complete computation–evidence–interpretation chain from source page 13. Implementation correctness and model suitability require different evidence.

A trustworthy report retains inputs, meanings, units, assumptions, and tests alongside its final value. Another reader should be able to reconstruct why the value follows and what it does not establish. This is the organizing principle behind the entire review, from elementary expressions to class interfaces and algorithm choice.

1.2. Arithmetic depends on operands as well as operators

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 9, 11.

For ordinary numeric values, `+`, `-`, and `*` perform addition, subtraction, and multiplication. Python requires explicit multiplication: write `2 * x`, not the algebraic shorthand $2x$. The power operator is `**`; the caret `^` is not Python exponentiation. The difference becomes important again when comparing languages in Module 13.

Expression	Result	Interpretation
`7 / 2`	`3.5`	True division.
`7 // 2`	`3`	Floor of the quotient, not ordinary rounding.
`7 % 2`	`1`	Remainder; useful for divisibility or cycles.
`2 ** 3`	`8`	Exponentiation.
`"7" + "2"`	`"72"`	String concatenation, not arithmetic.
`"3" * 2`	`"33"`	Repeat a string twice.

The familiar formula $E_k = \frac{1}{2}mv^2$ becomes `0.5 * mass_kg * velocity_m_s**2`. A correct translation preserves which quantity is squared. Parentheses make grouping explicit, particularly when a denominator contains a sum or when a formula is unfamiliar. The practice question later in this module checks how parentheses alter `2 + 3 * 4`.

The same appearance does not imply the same type. An integer `12` can represent a count; a floating-point `12.0` can represent an approximate numeric quantity; and `"12"` is text that might be an asset identifier. A Boolean answers a stated yes-or-no question. `None` commonly marks absent information or the absence of a returned result under a documented interface. It is not a numerical zero.

1.3. Assignment happens when the statement executes

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 10.

The right-hand side of an assignment is evaluated first. The resulting object is then associated with the name on the left. Python does not remember an algebraic dependency and automatically reevaluate it whenever an earlier input name changes.

```
# Store the area for the radius at this particular assignment.
radius = 2.0
area = 3.14 * radius**2
radius = 3.0  # Rebind radius; the earlier area is not recalculated.
print(radius, area)
```

Code 1.1: An earlier result does not automatically follow later reassignment.

```
3.0 12.56
```

When `area` is assigned, the radius is 2.0. The calculated value is $3.14 \times 2^2 = 12.56$. Reassigning `radius` to 3.0 changes that name, not the stored result associated with `area`. To calculate the area for the new radius, the calculation must execute again.

The numerical value 3.14 is the illustrative approximation used in the source; it is not an assertion that the circle constant is exact. The point of the example is execution order, not geometric accuracy. Meaningful names and unit-bearing suffixes help a reader distinguish quantities, but names alone do not validate their contents.

1.4. Input, output, and intended meaning

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 11.

When `input()` succeeds it returns text. A string that looks numerical remains text until a conversion is requested. Decide whether the intended quantity is a count, a measurement, or an identifier before choosing `int()`, `float()`, or no conversion. Converting an identifier can discard meaningful formatting, such as leading zeros.

Similarly, `print()` is communication, not verification. A formatted decimal result may look precise without being accurate. Read the value together with its unit, source, and acceptance rule. The distinction between displaying a value and returning a reusable result is developed in Module 4.

1.5. A complete stress calculation with units

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 12.

The source supplies force 10,000 N and area 500 mm^2. Since one millimetre is 10^{-3} metres, one square millimetre is 10^{-6} square metres. Therefore

$$A = 500 \times 10^{-6} = 5 \times 10^{-4}\ \text{m}^2, \qquad \sigma = \frac{10{,}000}{5 \times 10^{-4}} = 2 \times 10^{7}\ \text{Pa} = 20\ \text{MPa}.$$

```python
# Inputs: force in N and a positive cross-sectional area in mm^2.
force_n = 10_000.0
area_mm2 = 500.0
area_m2 = area_mm2 * 1e-6  # Square the length conversion factor.
stress_pa = force_n / area_m2  # N/m^2 gives Pa.
stress_mpa = stress_pa / 1e6  # Convert Pa to MPa for the report.
print(f"Stress = {stress_mpa:.1f} MPa")
```

Code 1.2: Convert the supplied area before calculating and reporting stress.

```
Stress = 20.0 MPa
```

Each variable denotes a particular representation of a physical quantity. `area_mm2` and `area_m2` refer to the same physical area in different units, but their numerical values differ. The output conversion from Pa to MPa is another separate arithmetic operation.

An independent hand calculation checks the absolute scale. A trend test, such as doubling force and expecting twice the stress, provides additional evidence but would not expose every constant unit multiplier error. Both kinds of evidence are useful.

Engineering check

This calculation assumes uniform axial stress. It neither models every possible failure mode nor supplies a design standard. A pass/fail decision needs a stated criterion, and a real engineering judgment may require a more appropriate model.

1.6. From power to energy: the same unit discipline

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 16–17.

A 2 kW heater operating for 30 minutes uses 1 kWh under a constant-power input model. The factor of 60 changes the numerical representation of duration; it is not an arbitrary correction applied to make a result look reasonable. The following labelled completion makes the question's unit chain executable.

```
# Worked solution to M01-Q2: the model uses constant input power.
power_kw = 2.0
operating_time_min = 30.0
operating_time_h = operating_time_min / 60.0
energy_kwh = power_kw * operating_time_h  # kW times h gives kWh.
print(f"Input electrical energy = {energy_kwh:.1f} kWh")
```

Code 1.3: Worked completion of the heater question; useful-heat efficiency is not assumed.

```
Input electrical energy = 1.0 kWh
```

Useful heat delivered to a particular body is a different question from electrical energy supplied. The review deliberately does not invent an efficiency, a loss rate, or a heating model.

1.7. Practice: attempt before consulting the solutions

1.7.1. M01-Q1: Values, types, and precedence

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 14.

Write all four printed lines. Identify the expression that repeats characters rather than multiplying a number, and explain which parentheses change the result.

```
# Predict all four lines before running the program.
print("7" + "2", int("7") + 2)  # Text joining versus numeric addition.
print(7 / 2, 7 // 2, 7 % 2)  # Quotient, floor quotient, remainder.
print("3" * 2, 3 * 2)  # Character repetition versus multiplication.
print(2 + 3 * 4, (2 + 3) * 4)  # Parentheses change the grouping.
```

Code 1.4: Practice M01-Q1. Predict before running.

Worked solution: page 5

1.7.2. M01-Q2: A unit error can look plausible

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 16.

A 2 kW heater operates for 30 minutes. A program evaluates `energy_kwh = 2 * 30`. Explain the defect, calculate the correct kWh value, and state one assumption.

Worked solution: page 5

1.8. Worked solutions and reasoning

M01-Q1: worked solution

```
72 9
3.5 3 1
33 6
14 20
```

The first expression joins the character strings `"7"` and `"2"`, giving `"72"`. The second converts `"7"` to the integer 7 before adding 2. In the next line, true division, floor division, and remainder answer different questions. The third line repeats a one-character string twice but multiplies two integers in the neighboring expression. Finally, multiplication precedes addition unless the parentheses require the sum to be evaluated first.

Takeaway

An operator is not understood until its operands and their types are known.

Return to question M01-Q1 (page 4).

M01-Q2: worked solution

The expression multiplies kilowatts by minutes, so its value represents 60 kW min, not 60 kWh. The required time is $30/60 = 0.5$ h, giving $E = 2 \times 0.5 = 1$ kWh. This assumes constant 2 kW electrical input power over the stated duration. It does not determine how much useful heat reaches a particular object; that would require an additional efficiency or heat-transfer model. Renaming a variable cannot perform a unit conversion.

Takeaway

Always derive the output unit from the input units and the actual arithmetic.

Return to question M01-Q2 (page 4).

1.9. Module checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 18.

Before proceeding, explain a problem, model, algorithm, and program using one example. Trace reassignment without imagining automatic recalculation. Identify a type from the stored object rather than its appearance. Finally, modify one physical input and predict both the numerical trend and the output unit before running the code.

2 Decisions and repetition

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 19–32.

After reviewing this module

Translate a verbal rule into explicit Boolean expressions; select a branch structure deliberately; trace range candidates and actual loop visits; distinguish counters from physical state; and explain why a loop stops and whether it succeeds.

2.1. One classification or several independent checks?

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 20–22.

An `if/elif/else` chain chooses its first true condition and skips the remaining alternatives in that chain. This is appropriate for a single category, such as too low, normal, or too high. Separate `if` statements can all execute; this is appropriate when several independent warnings are allowed. Choosing between the two structures is part of the algorithm, not merely a preference in formatting.

Python uses indentation to establish the blocks. Moving a warning message out of its branch makes it run unconditionally whenever execution reaches that line. A script can therefore display a misleading message without raising any exception. When tracing a decision, mark the block first, then evaluate its condition using the values that exist at that point.

For increasing thresholds, consider the narrow lower ranges before wider ones. The source classification assigns exactly 2.0 to the acceptable-by-stated-rule category:

```
# These are illustrative categories, not a certified design standard.
fos = 2.0
if fos < 1.0:
    status = "failure expected"
elif fos < 2.0:
    status = "insufficient margin"
elif fos < 4.0:
    status = "acceptable by stated rule"  # Includes exactly 2.0.
else:
    status = "conservative"
print(status)
```

Code 2.1: A factor-of-safety classification with deliberately chosen boundary behavior.

```
acceptable by stated rule
```

At 2.0 the first two comparisons are false and `fos < 4.0` is true. The final category follows from all those earlier decisions, not just the condition on the selected line. Test just below, exactly at, and just above each threshold. The labels are teaching categories, not a design code.

2.2. Boolean expressions should express the actual rule

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 21.

Use `and` when both conditions must hold, `or` when either is sufficient, and `not` when reversing a Boolean condition. For example, a system might require both `temperature_ok` and `pressure_ok`. A warning may be required if `too_hot` or `too_high` holds. The chained comparison `20 <= value <= 30` accepts both endpoints.

Common mistake

`unit == "pa" or "kpa"` does not compare the unit twice. It evaluates a comparison and then, if needed, a nonempty string, which is truthy. Write both comparisons explicitly, or use `unit in ("pa", "kpa")` for membership of the stated permitted collection.

Distinguish `=` (assignment) from `==` (equality comparison). A condition is a question about values already available; an assignment changes which object a name refers to. A good rule also identifies units and assumptions: numerical comparison cannot detect that one pressure was supplied in Pa and another in kPa.

2.3. A range gives candidates, not a guarantee of visits

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 23.

`range(start, stop, step)` generates integers beginning at the start and progressing by the step while staying on the required side of the excluded stop. Omitting the start uses zero; omitting the step uses one. A zero step is invalid. An empty range gives no loop-body passes.

```
# list() is added here only to display all generated integers.
print(list(range(4)))
print(list(range(1, 5)))
print(list(range(2, 9, 2)))
print(list(range(5, 0, -2)))  # Negative step: stop is still excluded.
print(list(range(3, 3)))  # Empty sequence; no loop passes.
```

Code 2.2: Display the source range examples as complete lists of integers.

```
[0, 1, 2, 3]
[1, 2, 3, 4]
[2, 4, 6, 8]
[5, 3, 1]
[]
```

The added `list()` calls simply make each range visible for this demonstration. A `for` loop need not convert a range to a list first. Furthermore, a `break` can end the loop before all candidates are visited. A `continue` skips the remainder of one pass but permits later candidates. These distinctions explain the exact execution path in question M02-Q1.

2.4. Initialization and update define the state history

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 24.

A counter usually starts at zero and increments when a specified event occurs. An accumulator starts at an appropriate neutral value and combines included values. A physical state may instead update according to a model equation, such as volume added by a known flow over a time step.

Role	Initial value	Update and meaning
Warning count	`count = 0`	Add one for each warning, not each numerical unit.
Total load	`total_n = 0.0`	Add the next included load in N.
Elapsed time	`time_s = 0.0`	Add the step duration in s.
Filled volume	`volume_l = 0.0`	Add `flow_l_s * step_s` in L.

Initialization normally belongs before the loop. Resetting a subtotal inside the body discards the accumulated history. A counter that reports three warnings is not a total temperature, pressure, or elapsed duration. Name the state so that its meaning is visible, and include it in a trace table when investigating a defect.

2.5. A while loop requires progress and a stopping explanation

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 25.

A `while` condition is tested before each pass. Its body may execute zero times if the condition is initially false. To justify termination, identify what changes and why those changes can reach a stopping state. A work budget provides an additional escape, but reaching it does not prove that the desired numerical condition was achieved.

The source shows a pattern using `error`, `candidate`, `step`, `target`, `tolerance`, and `maximum`; it explicitly does not define them. The following worked completion uses $x^2 = 10$, a starting candidate of 3.0, step 0.1, tolerance 0.001, and a budget of ten updates. These concrete setup values are additions for an executable illustration.

```
# The slide is a pattern. These inputs complete it for a small test.
target = 10.0
candidate = 3.0
step = 0.1
error = abs(candidate**2 - target)
tolerance = 0.001
maximum = 10
attempts = 0
while error > tolerance and attempts < maximum:
    candidate = candidate + step  # Move to the next grid candidate.
    error = abs(candidate**2 - target)
    attempts += 1  # Every pass advances the work counter.
# Reaching a finite budget is not the same as meeting the tolerance.
print("Accepted:", error <= tolerance)
print("Updates:", attempts)
```

Code 2.3: Labelled completion of the source stopping-pattern excerpt.

```
Accepted: False
Updates: 10
```

The two conditions are joined with `and`. Repetition continues only while the result still fails the tolerance and the budget remains. After the loop, the acceptance condition is checked again. This separates termination from convergence. Module 3 explains why this particular candidate grid does not include a satisfactory approximation.

2.6. A full time step can cross a target

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 30–31.

A tank fills at 3 L/s from an initial zero volume. With one-second full steps, the states are 3, 6, 9, and 12 L at 1, 2, 3, and 4 s. The condition "below 10 L" is false only after the fourth update. Under constant flow, the exact model time is 10/3 s.

```python
# Constant flow of 3 L/s; observe the state after each full 1 s step.
volume_l = 0.0
time_s = 0.0
flow_l_s = 3.0
step_s = 1.0
target_l = 10.0
while volume_l < target_l:
    volume_l += flow_l_s * step_s  # State update with compatible units.
    time_s += step_s
print(f"Simulated state: {volume_l:.1f} L at {time_s:.1f} s")
print(f"Exact model time: {target_l / flow_l_s:.4f} s")
```

Code 2.4: Worked completion of the source tank-filling question.

```
Simulated state: 12.0 L at 4.0 s
Exact model time: 3.3333 s
```

The simulator reports the first sampled state beyond the target. Its result is not inconsistent with its step rule, but its event time is approximate. Clipping the volume after the update does not repair the duration. This is a general engineering lesson: changes in output formatting or post-processing do not substitute for a suitable numerical model.

2.7. Practice: attempt before consulting the solutions

2.7.1. M02-Q1: Trace break and continue

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 26.

Write the final total, list the visited loop values, and explain whether value 6 is processed.

```python
# The accumulator is initialized once, before the loop.
total = 0
for value in range(1, 7):
    if value == 2:
        continue  # Skip addition, but proceed to the next iteration.
    if value == 5:
        break  # Stop the entire loop before adding 5.
    total += value
print(total)
```

Code 2.5: Practice M02-Q1. Predict before running.

Worked solution: page 10

2.7.2. M02-Q2: Repair a classification

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 28.

The rule accepts $20 \leq T \leq 30$. Explain why testing "if $T < 30$: normal" before "elif $T < 20$: too low" fails. Give three boundary tests.

Worked solution: page 10

2.7.3. M02-Q3: Stopping is not exact event timing

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 30.

A tank starts at 0 L and fills at 3 L/s. A loop adds 3 L and 1 s on each pass while volume is below 10 L. What final state is reported? What is the exact constant-flow filling time?

Worked solution: page 11

2.8. Worked solutions and reasoning

M02-Q1: worked solution

```
8
```

The visited values are 1, 2, 3, 4, and 5. After those visits the totals are 1, 1, 4, 8, and 8. The `continue` at 2 skips the addition for that pass. The `break` at 5 exits before the addition and before another value can be requested. Thus 6 belongs to the range but is never visited. Distinguish the generated candidate sequence from the executed path.

Takeaway

A loop trace must include values that trigger control-flow changes, even when they do not contribute to the sum.

Return to question M02-Q1 (page 9).

M02-Q2: worked solution

For any $T < 20$, the first test $T < 30$ is already true; Python selects that branch and never reaches the later low-temperature test. At $T = 30$, the first test is false even though the rule accepts equality. Use `if T < 20`, then `elif T <= 30`, and finally `else`. A useful minimum set is 19.9 (too low), 20.0 (normal), and 30.0 (normal). Add 30.1 to exercise the high branch and neighboring values when reviewing all boundary transitions.

Takeaway

The selected branch depends on both comparison symbols and their order.

Return to question M02-Q2 (page 10).

M02-Q3: worked solution

The states after successive full steps are $(3\text{ L}, 1\text{ s})$, $(6, 2)$, $(9, 3)$, and $(12, 4)$. The next condition is false, so the reported state is 12 L at 4 s. Under the constant-flow model, $t = V/Q = 10/3 \approx 3.3333$ s. The algorithm observes the target only after full steps. Clipping the displayed volume from 12 L to 10 L without adjusting the duration does not make 4 s an exact event time.

Takeaway

An algorithm can terminate correctly while reporting a time-resolution-limited event estimate.

Return to question M02-Q3 (page 10).

2.9. Module checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 32.

Translate the English rule explicitly, choose a chain or independent checks, and trace every changed variable. Exercise zero, one, and several loop passes. Explain whether the loop stopped because it succeeded, exhausted its candidates, or reached a safeguard. A program that executes without an exception can still implement the wrong rule.

3 Strings and numerical search

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 33–50.

After reviewing this module

Select characters and substrings correctly; preserve raw text and identifiers; state parsing assumptions; distinguish candidate resolution from tolerance; trace bracket-preserving bisection; and report an approximation with its evidence and limitations.

3.1. An index selects a character; a slice selects a sequence

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 34.

A string is an ordered sequence. For `s = "PUMP-107A"`, the forward positions are 0 through 8. Negative index −1 selects the last character. A slice `s[start:stop:step]` selects a subsequence with the stop excluded. The order and step determine which positions are visited.

Expression	Result	Selected meaning
`s[0]`	`"P"`	First character.
`s[-1]`	`"A"`	Last character.
`s[:4]`	`"PUMP"`	Positions 0, 1, 2, 3.
`s[5:]`	`"107A"`	Position 5 through the end.
`s[::2]`	`"PM-0A"`	Positions 0, 2, 4, 6, 8.
`s[::-1]`	`"A701-PMUP"`	All characters in reverse order.

Source clarification

The revised review explicitly corrects an earlier Week 3 shortcut-table result. The correct value of `"PUMP-107A"[::2]` is `"PM-0A"`, not `"PMP0A"`. These notes retain that visible correction and the selected-index explanation.

Use a written position row before attempting a complicated slice. Direct indexing outside the sequence raises an error, while out-of-range slice bounds are clipped. The simple length shortcut $b - a$ applies only to ordinary in-range, ordered bounds with step one; Module 5 returns to that qualification for lists.

3.2. Transform text without silently changing its meaning

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 35.

Strings are immutable. Methods such as `strip()` and `upper()` return transformed strings; they do not replace characters of the original string object. Store the returned value under a meaningful name, or deliberately rebind the original name when that is appropriate.

```python
# Keep the raw identifier text unchanged for traceability.
raw = " Pump-007 "
clean = raw.strip()  # Remove surrounding whitespace only.
upper = clean.upper()
has_prefix = upper.startswith("PUMP-")
position = upper.find("-")
pieces = upper.split("-")
joined = ":".join(pieces)  # Retain the leading zeros in the ID.
# Output statements are added to make all intermediate values visible.
print(repr(raw), clean, upper)
print(has_prefix, position, pieces, joined)
```

Code 3.1: Preserve raw text while exposing each transformation.

```
' Pump-007 ' Pump-007 PUMP-007
True 4 ['PUMP', '007'] PUMP:007
```

`find()` returns the position of the first match or -1 for absence. That sentinel is not a successful location: using it as an index without checking could select the last character. `split()` creates pieces around a separator, and `join()` combines string pieces using the receiving string as a separator.

A leading zero, a hyphen, or a capitalization convention may identify a component. The source retains `007` as text rather than converting it into the integer 7. Normalization is justified only by a stated rule. A changed unit label likewise does not convert the numerical value into the new unit.

3.3. Parsing is an agreement about a record format

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 36.

The teaching message contains three comma-separated fields: identifier, numerical text, and unit. Unpacking the result of `split(",")` into three names assumes exactly three fields. Too few or too many fields violates that structural assumption. A numerical conversion can fail independently of the number of fields.

```python
# This first parser assumes exactly three comma-separated fields.
raw = " T-07 , 23.50 , C "
name, text, unit = raw.split(",")
name = name.strip()  # Preserve the ID as text, including its zero.
value = float(text)  # Numeric conversion is separate from unit checking.
unit = unit.strip()
print(name, value, unit)
print(name[-2:])
```

Code 3.2: An intentionally simple parser with stated structural assumptions.

```
T-07 23.5 C
07
```

The identifier remains a string, so its zero is preserved. The numeric field becomes a float. The code cleans the unit but does not yet verify that it is compatible with the intended calculation. Module 14 supplies the fuller parser: it checks field count, nonempty ID, exact unit, successful

conversion, and finiteness separately.

3.4. Guess-and-check is a procedure, not intuition

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 37–38.

A numerical search needs a candidate rule, a model evaluation, an acceptance test, an update, and a safeguard or finite candidate range. Each part answers a different question. For an exact integer cube root, candidates can be integer magnitudes in increasing order. Once the candidate cube reaches or exceeds the target magnitude, an equality test distinguishes an exact integer root from an overshoot.

```
# Enumerate magnitudes; this is an integer search, not a real-root solver.
target = -125
candidate = 0
while candidate**3 < abs(target):
    candidate += 1  # The candidate moves toward or beyond the target.
if candidate**3 == abs(target):
    if target < 0:
        candidate = -candidate  # Restore the sign for an odd root.
    print("Integer cube root:", candidate)
else:
    print("No integer cube root")  # No claim about real roots is made.
```

Code 3.3: Worked completion of the exact integer cube-root procedure described in the source.

```
Integer cube root: -5
```

A failure here means that no exact integer cube root was found in the systematic enumeration; it does not mean that a real cube root does not exist. Searching a magnitude first and restoring the sign uses the mathematical behavior of an odd power.

For incremental real-valued search, the step h sets the candidate grid and a residual tolerance ε_f sets acceptance. They are not interchangeable. Approximately L/h trials are needed to cover an interval of length L, but a coarse grid may skip every acceptable point. A stopped search should report its candidate, residual, tolerance, trial count, and stopping reason. Exhausting a work budget is not convergence.

3.5. Bisection retains a root-containing interval

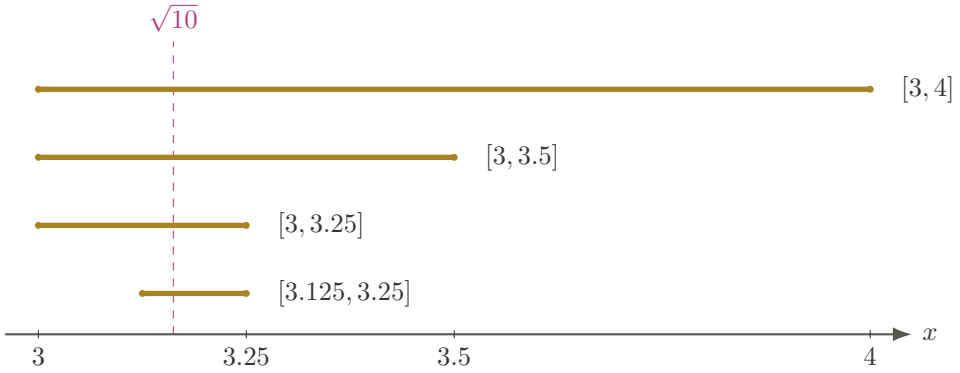
Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 39–40, 43–44.

Write the engineering target as $f(x) = 0$. The bisection argument assumes continuity on the bracket. Exact endpoint roots can be handled first; otherwise the endpoints must have opposite residual signs. Compute the midpoint and retain the half with the appropriate sign change. In exact arithmetic, each such update halves an interval that still contains a root.

For $f(x) = x^2 - 10$ on $[3, 4]$, the left residual is negative and the right residual positive. The following trace records residuals, not just candidate values.

a	b	m	$f(m)$	Retained bracket
3	4	3.5	2.25	$[3, 3.5]$
3	3.5	3.25	0.5625	$[3, 3.25]$
3	3.25	3.125	-0.234375	$[3.125, 3.25]$
3.125	3.25	3.1875	0.16015625	$[3.125, 3.1875]$

Both endpoint values in this example are positive numbers. The decision depends on the signs of their *residuals*, not whether the endpoint itself is less than zero. A general bisection routine must evaluate the supplied function rather than blindly compare an input variable with a target.




```python
# x^2 - 10 is increasing and continuous on this positive bracket.
left, right = 3.0, 4.0
for _ in range(10):
    mid = (left + right) / 2
    if mid**2 < 10:
        left = mid  # The negative-residual endpoint moves right.
    else:
        right = mid  # The nonnegative-residual endpoint moves left.
estimate = (left + right) / 2  # Use the updated bracket midpoint.
print("Estimate, half-width:", estimate, (right - left) / 2)
```

Code 3.4: The source's specialized ten-update calculation.

```
Estimate, half-width: 3.16259765625 0.00048828125
```

This version deliberately exploits the increasing function $x^2 - 10$ on a positive bracket. It is not a general-purpose function. Its loop performs ten completed updates and then calculates the midpoint of the updated bracket.

3.6. Residual tolerance and input uncertainty differ

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 41.

A residual condition $|f(x)| \leq \varepsilon_f$ measures agreement with the output target. The tolerance has the residual's unit. It does not automatically bound the difference between the current input and a true root. The sensitivity of the model can affect how an output residual relates to an input error.

A retained bracket gives a different argument. If a root x^* is in $[a, b]$ and $m = (a + b)/2$, then

$$|m - x^*| \leq \frac{b - a}{2}.$$

After k completed halvings of initial width L_0, the width is $L_k = L_0/2^k$, and the midpoint bound is $L_0/2^{k+1}$. Thus a target midpoint uncertainty ε_x requires

$$\frac{L_0}{2^{k+1}} \leq \varepsilon_x.$$

The initial midpoint may already meet the requirement, in which case no halving is needed. Always state whether the stopping criterion refers to width or half-width. In these notes the ten-update example has width 1/1024 and half-width 1/2048.

3.7. What a bracket argument cannot establish

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 42.

A sign change across a discontinuity is not a root certificate. The function $1/x$ changes sign across zero but has no zero there and is not continuous across that point. Conversely, the touching root of $(x-1)^2$ has no sign crossing. An interval containing two crossings can also have equal endpoint signs. These are limitations of the sign-change test, not proof that such equations have no roots.

A maximum iteration count or floating-point stagnation must be reported as failure to meet the requested condition, unless a separate acceptance test succeeds. The mathematical width guarantee is an exact-arithmetic statement; practical computations also have representational limitations. Finally, even an accurately approximated root of a cooling or projectile equation inherits the model's assumptions.

3.8. Practice: attempt before consulting the solutions

3.8.1. M03-Q1: Trace strings without changing the original

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 45.

Write all three output lines. Explain whether `raw` is modified and identify the characters selected by `[6:9]`.

```
# Both methods return new strings; raw retains its original characters.
raw = " valve-204b "
code = raw.strip().upper()
print(code[:5], code[6:9], code[-1])  # Stop indices are excluded.
print(code[1:8:2])  # Select indices 1, 3, 5, and 7.
print(repr(raw))  # Make the remaining surrounding spaces visible.
```

Code 3.5: Practice M03-Q1. Predict before running.

Worked solution: page 17

3.8.2. M03-Q2: Turn bracket width into an error bound

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 47.

A valid bracket has initial width 1. After eight completed halvings, what are its width and midpoint error bound? How many halvings suffice for midpoint error at most 10^{-3}?

Worked solution: page 17

3.8.3. M03-Q3: No acceptable grid point is not no root

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 49.

Search candidates $3.0, 3.1, 3.2, \ldots, 4.0$ for $|x^2 - 10| < 0.001$. Why does the search fail despite $\sqrt{10} \in [3, 4]$? Propose a better approach.

Worked solution: page 17

3.9. Worked solutions and reasoning

M03-Q1: worked solution

```
  VALVE 204 B
  AV-0
  ' valve-204b '
```

The cleaned uppercase identifier is `VALVE-204B`. Indices 6, 7, and 8 hold `2`, `0`, and `4`. The stepped slice uses indices 1, 3, 5, and 7, producing `AV-0`. Both methods return new strings; the original `raw` still contains its spaces and lowercase text. `repr(raw)` displays those spaces between quotation marks.

Takeaway

Write the index row before calculating a complicated slice.

Return to question M03-Q1 (page 16).

M03-Q2: worked solution

After k completed halvings the width is 2^{-k}, and the updated midpoint is at most half that width from a retained root. At $k = 8$, width $= 1/256 = 0.00390625$ and midpoint bound $= 1/512 = 0.001953125$. At $k = 9$, the bound is $1/1024 = 0.0009765625 \leq 10^{-3}$. Nine halvings therefore suffice. The claim assumes the root remains bracketed, the function is continuous on the relevant interval, and arithmetic is exact. The units of these bounds are the units of the input variable.

Takeaway

Check whether a tolerance refers to full width, half-width, or the residual.

Return to question M03-Q2 (page 16).

M03-Q3: worked solution

The residual at 3.1 is -0.39, and at 3.2 it is 0.24. Neither is close enough, and the root lies between them. On this interval the residual increases, so the other grid values are even farther from zero. The failure describes the finite candidate grid, not the mathematical equation. Bisection can retain a sign-changing continuous bracket and reduce its width to a stated input-space requirement. A finer grid is another possible experiment, but its cost and stopping condition must still be stated.

Takeaway

Report the method and stopping reason with the numerical result.

Return to question M03-Q3 (page 17).

4 Functions and program design

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 51–63.

After reviewing this module

Explain function definition and invocation, trace parameter binding and scope, separate returned values from displays and mutation, and design focused components whose interfaces preserve engineering units and meaning.

4.1. Define the tool, then invoke it

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 52.

A function definition associates a name with a reusable procedure. The body is not executed merely because it has been written. A call evaluates its argument expressions, binds the results to local parameter names, executes the body, and returns control to the caller with a result.

```
# The caller supplies force in N and positive area in m^2.
def calculate_stress(force_n, area_m2):
    """Return uniform axial stress in Pa."""
    return force_n / area_m2  # A calculation result, not printed text.

result_pa = calculate_stress(10.0, 2.0)
print(result_pa)  # Added display; hand expectation is 5 Pa.
```

Code 4.1: A small engineering function and a hand-checkable call.

```
5.0
```

The two argument values are 10.0 and 2.0. Inside the call they are known as `force_n` and `area_m2`. Division produces 5.0 Pa, and the return statement supplies that number to `result_pa`. A function call can appear in a larger expression because it evaluates to its returned object.

4.2. Parameters, arguments, and defaults

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 53.

A parameter is the local name in the definition; an argument is the object supplied in a call. Positional arguments bind by order. Keyword arguments make the parameter association visible at the call site. In engineering code this can prevent a reviewer's reliance on memorized argument order, but it does not enforce units.

A default, such as a required factor of safety of 2.0 in a teaching classifier, supplies a documented choice when a caller omits that argument. An explicit argument replaces that choice for the current call. A default is not a universal engineering standard. The same principles explain the difference between the outer `gain` name and the local default in mixed question X02.

4.3. Returning, printing, and mutating are different actions

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 54.

A `return value` statement ends the current call and makes that object available to the caller. `print(value)` displays information, and the print call itself returns `None`. A function that reaches its end without returning a value also returns `None`. Displaying a number does not deliver that number as reusable data.

A function can additionally mutate a supplied list or object through a shared reference. That effect is independent of its returned value. For example, a list append changes the list and returns `None`. A function can return a useful scalar while also modifying an input; its contract must say whether that side effect is intended.

A clear calculation layer returns data. A reporting layer constructs text or displays it according to its own contract. The caller then chooses whether to print, retain, save, or further process the result. This separation makes testing easier because numerical behavior is not hidden inside terminal output.

```
# Repair the interface without changing E = P*t or the default time.
def energy(power_w, time_s=2):
    """Return energy in J for the supplied constant power and duration."""
    energy_j = power_w * time_s
    return energy_j  # The caller can now reuse this number.

result = energy(10)
print(result)
```

Code 4.2: Worked repair of the printing-only energy function in M04-Q1.

```
20
```

The deliberately faulty version remains in the practice question. Here the repair changes only the interface's output behavior; it does not change $E = Pt$.

4.4. Local names and shared objects

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 55.

A parameter belongs to a particular call. Giving it the same spelling as a caller variable does not create one shared variable name. Assignment to the local name changes that local binding. However, if the parameter refers to a mutable object, operations on that object can still be visible through the caller's reference.

```python
value = 10  # A name in the outer scope.

def double(value):
    # The parameter is local to this call; integers are immutable.
    value = 2 * value
    return value

result = double(value)
print(value, result)
```

Code 4.3: Trace the outer binding, local parameter, and returned result separately.

```
10 20
```

The outer `value` remains 10. The local parameter initially refers to that integer value and is rebound to 20 during the call. The returned 20 is then associated with the outer name `result`. This explanation is about names and objects; the statement that local assignment does not rebind a caller name must not be misread as a promise that no mutable argument can ever change.

4.5. Decomposition and abstraction support each other

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 56–57.

Decomposition divides a larger task into responsibilities: parse, validate, convert, calculate, classify, and report. A useful function does one meaningful job whose behavior can be checked independently. Abstraction means using that function through its name and contract without repeating its internal algorithm in the calling code.

Neither principle means hiding information needed for correctness. A usable interface states input meanings and units, permitted ranges, returned meaning and accuracy, mutation behavior, and expected failures. A docstring records the promise; it does not enforce the promise. Explicit validation, correct implementation, and tests supply separate evidence.

Component	Input agreement	Output agreement
Area conversion	Finite positive area in mm^2.	Same area expressed in m^2.
Stress calculation	Force in N and positive area in m^2.	Signed uniform axial stress in Pa.
Classification	Value and ordered limits in one unit.	Category with a specified equality rule.
Report	Result, units, criterion, assumptions.	Readable information without unintended mutation.

Individually correct functions can be connected incorrectly. An area-conversion test does not prevent a caller from passing the unconverted input to the stress function. Retain an integration test: 10,000 N and 500 mm^2 must produce 20 MPa through the complete chain.

4.6. A return inside a loop changes the algorithm

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 62–63.

The `return` keyword ends the function call, not just the current loop pass. If a function is meant to sum all loads, returning after the first addition violates that requirement. Trace the location of the return and test more than one element, because a singleton case can accidentally appear correct.

```
# All loads contribute before a result is returned.
def total_load(loads_n):
    total = 0.0  # Also defines the empty-list result.
    for load in loads_n:
        total += load
    return total  # Same indentation as the loop header.

print(total_load([2.0, 3.0, 4.0]))
print(total_load([]))
```

Code 4.4: Worked repair of the premature-return exercise.

```
9.0
0.0
```

The repaired return occurs after every requested load has been included. Its placement also makes the empty-list result explicit through the initialization. That result is appropriate only when the contract defines an empty sum as zero; a different function, such as a mean, needs a different empty-input policy.

4.7. Practice: attempt before consulting the solutions

4.7.1. M04-Q1: What does the caller store?

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 58.

Predict both printed lines. Identify the default parameter value. Explain the repair required if later code must use the energy in arithmetic.

```
# Intentional design defect: a numeric result is not returned.
def energy(power_w, time_s=2):
    energy_j = power_w * time_s  # Default time is 2 s.
    print(energy_j)

result = energy(10)
print(result)  # The caller receives None from this function.
```

Code 4.5: Practice M04-Q1. Predict before running.

Worked solution: page 22

4.7.2. M04-Q2: Expose a hidden dependency

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 60.

A global `area_m2 = 0.002` is read by `def stress(force_n): return force_n / area_m2`. Why is this interface incomplete? Give a better header and one independent test.

Worked solution: page 22

4.7.3. M04-Q3: Repair an early return

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 62.

Predict the printed result. Identify the values that never contribute. Where should `return` be placed?

```
# This function deliberately fails its all-loads summation contract.
def total_load(loads_n):
    total = 0.0
    for load in loads_n:
        total += load
        return total  # Wrong indentation: stops after the first load.

print(total_load([2.0, 3.0, 4.0]))
```

Code 4.6: Practice M04-Q3. Predict before running.

Worked solution: page 23

4.8. Worked solutions and reasoning

M04-Q1: worked solution

```
20
None
```

The call supplies 10 W and uses the default 2 s, so the internal calculation is 20 J. The internal `print` displays 20 but no value-returning statement is executed. The function therefore returns `None`, which is stored in `result` and printed next. Replace the internal display statement with `return energy_j`; let the caller print the returned number. This repairs the interface without changing the formula or the default.

Takeaway

Displayed output and the value delivered to a caller are separate observations.

Return to question M04-Q1 (page 21).

M04-Q2: worked solution

The result depends on an area that is absent from the parameter list. Changing that outer name can change the result of an apparently identical call. Use `def stress(force_n, area_m2):` and return their ratio. Document force in N, area in m^2, a positive area, and output in Pa. The independent test `stress(10.0, 2.0)` should return 5.0 Pa. Rejection tests are appropriate only when the function promises explicit validation; a documented caller precondition and an implemented check are not the same thing.

Takeaway

Inputs that determine the answer should be visible in the interface or explicitly documented configuration.

Return to question M04-Q2 (page 21).

M04-Q3: worked solution

```
2.0
```

The first pass adds 2.0 to the initial subtotal and then immediately returns 2.0. The remaining values, 3.0 and 4.0, are never processed. Place `return total` after the loop, at the same indentation as the loop header. The corrected result is 9.0. That placement also returns the initialized 0.0 for an empty input if the intended contract is the sum of all supplied loads.

Takeaway

Trace where a call ends, not just which arithmetic expression appears in the body.

Return to question M04-Q3 (page 22).

5 Lists, Tuples, and Data Integrity

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 64–78.

After reviewing this module

Choose a collection by its purpose; distinguish an index, a value, a returned result, and a side effect; trace aliases and shallow copies; and preserve the associations that give measurements meaning.

5.1. Choose a representation before processing it

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 65.

A list is an ordered, mutable collection. Its order may represent acquisition time, a catalogue order, or a sequence of design alternatives. A tuple is also ordered, but its element references cannot be replaced after creation. A tuple is useful for a fixed collection of associated fields, such as a time–temperature pair. Neither container automatically knows what its fields mean or which physical units they use.

A decision to use a tuple does not make every object inside it immutable. A tuple containing a list still refers to a mutable list. Likewise, choosing a list does not require the program to modify it: an input list can be protected by an explicit non-mutation contract.

Need	Suitable representation	Information to document
A growing series of readings	List of numbers	Unit, sample spacing, acquisition order.
One fixed coordinate or record	Tuple	Meaning and unit of each position.
Many associated observations	List of tuples	A consistent field order in every record.
A named lookup relationship	Dictionary	Key convention, value meaning, missing-key policy.

These are design choices, not a ranking from primitive to advanced. The simplest representation that preserves the task's information is often the best starting point.

5.2. Indices, lengths, and data windows

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 66.

For a nonempty sequence of length n, forward indices are 0 through $n-1$ and negative indices are $-n$ through -1. An index selects one object; a slice selects a new sequence. An empty list has no valid element index, even though obtaining its length or iterating over it is valid.

A slice `values[a:b]` excludes position b. Its length is $b-a$ only when the forward, unit-step bounds already satisfy $0 \le a \le b \le n$. Python normalizes negative and out-of-range bounds, so that formula must not be applied blindly. This qualification preserves the intended half-open-window idea without confusing it with every possible slice expression.

`enumerate(values)` supplies both index and value. Use it when location matters; use direct iteration when only each value is needed. If two lists represent paired measurements, equal lengths are necessary but not sufficient: the entries at a shared index must refer to the same observation.

5.3. Track mutation and return values independently

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 67.

A collection operation may change an object, return an object, do both, or do neither. These are separate questions. For a list, `append(x)` adds one object; `extend(items)` adds the elements of an iterable. Appending a list creates a nested element, whereas extending by that list adds its elements individually.

Operation	Effect on the list	Returned result
`a[i] = x`	Replace the selected element.	Assignment is a statement.
`a.append(x)`	Add one element at the end.	`None`.
`a.extend(items)`	Add each supplied element.	`None`.
`a.remove(x)`	Remove the first equal value.	`None`; absent value raises an error.
`a.pop(i)`	Remove the element at the position.	The removed object.
`a.sort()`	Reorder this list in place.	`None`.
`sorted(a)`	Leave this list's order unchanged.	A new sorted list.

Do not assign the result of `a.sort()` to the name that should hold the sorted list. That would bind the name to `None`. The mutation has already happened, but the assignment loses that name's reference to the list.

5.4. Aliasing, shallow copying, and nested data

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 68,72.

Assignment copies a reference, not an entire collection. After `alias = raw`, both names lead to the same list. A change through either name is visible through the other. By contrast, `raw.copy()` creates a new outer list with the same element references.

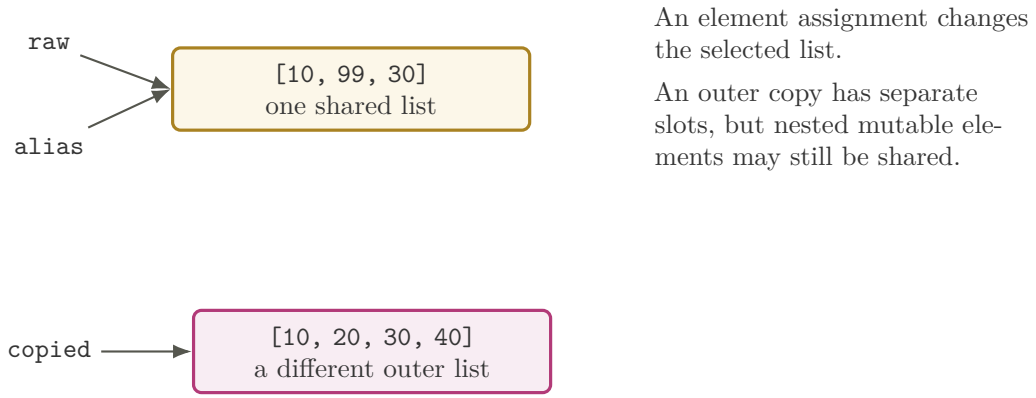


Figure 5.1.: The review's reference model: two names can select one list, while a shallow copy has separate outer slots. Inner mutable objects can still be shared.

```python
# Only the outer list is copied.
raw = [[1, 2], [3, 4]]
copied = raw.copy()
copied[0][0] = 9  # Mutate the inner list reached by both outer lists.
print(raw)
print(copied is raw, copied[0] is raw[0])
```

Code 5.1: An outer copy does not copy the nested lists.

```
[[9, 2], [3, 4]]
False True
```

The output must be explained at the level of the changed object. Replacing a slot in one outer list is different from changing an inner list to which both outer lists refer. A shallow copy is sufficient to protect the outer arrangement of a flat numeric list because numbers are immutable. It is not a general promise of independence for arbitrary nested records.

5.5. Filter without deleting from the collection being traversed

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 69.

Removing elements from a list during a forward iteration can shift unvisited elements into positions already passed by the iterator. A clearer pattern is to retain the raw list and build a separate list of accepted values.

```python
# Preserve raw order and keep the original list for inspection.
raw = [1, 2, 2, 3, 2, 4]
cleaned = []
for value in raw:
    if value != 2:
        cleaned.append(value)  # Append only the accepted values.
print(cleaned)  # Added output exposes the selected result.
print(raw)
```

Code 5.2: Build a filtered result while preserving raw readings.

```
[1, 3, 4]
[1, 2, 2, 3, 2, 4]
```

The condition is the stated selection rule. A value omitted from the result has not automatically been proved invalid: it may simply be outside the selected interval. Preserve the original record and the selection criterion so another reader can understand the transformation.

5.6. Tuples keep related fields together

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 70.

Packing groups related objects; unpacking assigns their references to matching names. A single-element tuple requires a comma. Parentheses alone can simply group an expression.

```python
# Commas distinguish tuples, including a one-element tuple.
empty = ()
singleton = (4.0,)
point = (2.0, 5.0)
x_m, y_m = point  # Unpack the two coordinate values.
samples = [(0.0, 20.0), (1.0, 22.0)]
for time_s, temperature_c in samples:
    print(time_s, temperature_c)  # A whole pair is kept together.
```

Code 5.3: Tuple syntax and unpacking retain the meaning of a record.

```
0.0 20.0
1.0 22.0
```

For a record `(time_s, temperature_c)`, each position has a fixed interpretation. A list of these records avoids the risk of sorting temperatures independently of timestamps. A returned pair from a function is similarly useful when the results belong together, provided its order is documented.

5.7. Summaries do not replace the measurements

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 71.

For $n > 0$, the arithmetic mean and root-mean-square value are

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad x_{\mathrm{RMS}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}.$$

Both return the original unit when the readings share one unit. Neither retains acquisition order. A mean can hide opposite deviations, and an RMS value describes magnitude rather than the signed average. Before calculating either, decide whether an empty input is rejected and how missing or rejected measurements are reported. Substituting zero for missing information changes the calculation and invents an observation.

5.8. Practice: attempt before consulting the solutions

5.8.1. M05-Q1: Trace operations and their return values

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 73.

Write all three output lines. Explain the list length immediately after `extend`. Which operation returns `None`?

```python
values = [10, 20, 30]
values[1] = 25  # Replace one item in the same list.
result = values.append(40)  # Mutates, but returns None.
values.extend([50, 60])  # Append each element of this iterable.
removed = values.pop()  # Return and remove the last element.
print(values)
print(result)
print(removed)
```

Code 5.4: Practice M05-Q1. Predict before running.

Worked solution: page 28

5.8.2. M05-Q2: Alias or independent outer list?

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 75.

Predict the three lists. Which assignment creates an alias? Why does appending to `copied` not affect `raw`?

```
raw = [10, 20, 30]
alias = raw  # Two names now refer to one list.
copied = raw.copy()  # A new outer list is made before the mutation.
alias[1] = 99
copied.append(40)  # Mutate the independent outer list.
print(raw)
print(alias)
print(copied)
```

Code 5.5: Practice M05-Q2. Predict before running.

Worked solution: page 29

5.8.3. M05-Q3: Preserve paired measurements

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 77.

Times are `[0, 1, 2]` s and temperatures are `[25, 20, 30]` °C. Why is sorting only temperatures misleading? Give a safer representation and one boundary check for a rate calculation.

Worked solution: page 29

5.9. Worked solutions and reasoning

M05-Q1: worked solution

```
[10, 25, 30, 40, 50]
None
60
```

Replacing index 1 keeps the length at three and changes 20 to 25. `append(40)` adds one item, making the length four; its return value is `None`. `extend([50, 60])` appends two separate items, making the length six. `pop()` removes and returns 60, leaving five items. The final list, saved append result, and removed element are three different results.

Takeaway

For each operation, record both the changed collection and the returned object.

Return to question M05-Q1 (page 27).

M05-Q2: worked solution

```
[10, 99, 30]
[10, 99, 30]
[10, 20, 30, 40]
```

`alias = raw` creates a second name for the original list. Replacing `alias[1]` therefore changes that shared list, so both `raw` and `alias` display 99. The copy was made before the replacement and has its own outer slots; it still contains 20 at position 1. Appending 40 changes only that separate outer list. This reasoning is sufficient for the flat numeric example, not a guarantee of independence for nested mutable elements.

Takeaway

Assignment is not copying, and an outer copy is not necessarily a deep copy.

Return to question M05-Q2 (page 28).

M05-Q3: worked solution

Sorting only the temperatures would associate 20°C with 0 s, although that measurement occurred at 1 s. Store complete pairs such as `[(0,25), (1,20), (2,30)]`. Sort those records only for an explicit purpose, retaining the identity of each pair. A finite-difference rate requires a nonzero time difference; repeated timestamps make division invalid. Decreasing timestamps also require a deliberate policy rather than a silent assumption of increasing time.

Takeaway

A set of correct numbers can still encode incorrect engineering evidence when associations are lost.

Return to question M05-Q3 (page 28).

5.10. Collection checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 78.

For any unfamiliar collection program, annotate each line with the object it reads, the object it changes, and the result it returns. Check zero, one, and several elements; first and last positions; aliases; and whether original ordering and units survive. Those checks are more informative than observing only the final scalar.

6 Recursion and Dictionaries

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 79–93.

After reviewing this module

Trace both recursive descent and returning results, justify termination, distinguish repeated subproblems from active call depth, and use dictionary keys to preserve meaning and handle absence explicitly.

6.1. Three obligations of a recursive function

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 80.

A recursive definition needs a valid input domain, a directly solvable base case, and a recursive rule whose smaller answer can be used to construct the current answer. A base case alone does not establish termination. Starting from a positive integer, a call using `n + 1` moves away from a base case at zero.

Write a measurable progress quantity before running the function. In factorial it is the nonnegative integer argument; in palindrome testing it is the string length. Every recursive route must reach an appropriate base case. A recursion-depth exception is a resource limit or a symptom to investigate, not evidence that the intended answer was reached.

6.2. Factorial: finish the return path

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 81.

For a nonnegative integer, $0! = 1$ and $n! = n(n-1)!$ for $n > 0$. The review implementation rejects Boolean and non-integer arguments explicitly, then rejects negative integers. This is a particular input contract; it is stricter than merely checking `isinstance(n, int)` because Python Boolean objects also satisfy that test.

```
# Every call must be a smaller nonnegative built-in integer.
def factorial(n):
    """Return n! for a nonnegative built-in integer; use small n."""
    if type(n) is not int:
        raise TypeError("n must be an integer, not bool")
    if n < 0:
        raise ValueError("n must be nonnegative")
    if n == 0:
        return 1  # Base case: the neutral value for multiplication.
    return n * factorial(n - 1)
```

Code 6.1: The review's factorial definition, including its input contract.

For `factorial(4)`, calls descend through 4, 3, 2, 1, and 0. The base returns 1. The waiting calls then return $1 \times 1 = 1$, $2 \times 1 = 2$, $3 \times 2 = 6$, and $4 \times 6 = 24$. Each frame retains the local value needed for its unfinished multiplication.

A trace that stops after writing the call sequence is incomplete. The result belongs to the return sequence, which unwinds in the reverse order. Statements before a recursive call execute during descent; statements after it execute when that deeper call has returned.

6.3. Fibonacci: two base cases and overlapping work

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 82–83,87.

This course uses

$$F(0) = 1, \qquad F(1) = 1, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

Do not substitute another indexing convention when checking the displayed answers. In this convention $F(4) = 5$ and $F(5) = 8$.

```
# The compact slide variant assumes the caller checked the domain.
def fibonacci_review(n):
    if n == 0 or n == 1:
        return 1  # Course base cases: 1, 1.
    return (fibonacci_review(n - 1)
            + fibonacci_review(n - 2))
```

Code 6.2: Compact Fibonacci recurrence under the stated nonnegative-integer precondition.

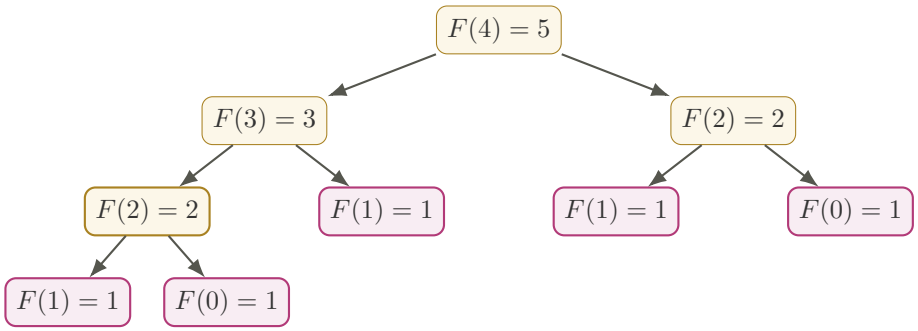


Figure 6.1.: All nine calls for $F(4)$, following the review's convention. Equal labels on different branches denote repeated computations, not one shared call.

A complete call tree includes both children of every non-base call. The longest active path is much shorter than the total number of calls across the tree. This distinction becomes central to the time–memory review in Chapter 11.

```
# Caller supplies a nonnegative integer; the course starts with 1, 1.
def fibonacci_iterative(n):
    previous, current = 1, 1
    for index in range(2, n + 1):
        # Both right-hand expressions use the previous pair of values.
        previous, current = current, previous + current
    return current

print(fibonacci_iterative(5))  # Added check: F(5) = 8.
```

Code 6.3: An iterative alternative retains the two values needed next.

```
8
```

The initial pair is $(1, 1)$. At sequence positions 2, 3, 4, and 5, the updated pairs are $(1, 2)$, $(2, 3)$, $(3, 5)$, and $(5, 8)$. The same mathematical sequence can therefore be evaluated without recomputing earlier terms. Agreement between the two programs is useful, but the small hand-calculated values supply the independent reference.

6.4. Recursion on strings

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 84.

A palindrome reads identically in both directions. The exact-text review function treats strings of length zero or one as palindromes. For longer strings, the outer characters must agree and the inner substring must be a palindrome.

```
# Each matching pair reduces the remaining string by two characters.
def is_palindrome(text):
    """Test the exact, case-sensitive characters of a string."""
    if len(text) <= 1:
        return True  # Empty and one-character strings are base cases.
    return (text[0] == text[-1]
            and is_palindrome(text[1:-1]))
```

Code 6.4: Recursive palindrome testing with a length-based progress argument.

For `"level"`, the reduced strings are `"eve"` and `"v"`. The singleton base case returns true, and the outer equality checks allow the true result to propagate outward. A mismatch stops the expression through short-circuit evaluation; the inner recursive call need not run when the first condition is false.

This implementation is case-sensitive and includes punctuation and whitespace. A separate normalization policy would be a different part of the specification. Do not silently remove characters to obtain the answer expected from an informal sentence-level definition of palindrome.

6.5. A dictionary represents associations, not numerical positions

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 85.

A dictionary associates unique keys with values. A key is a lookup label rather than an offset into a sequence. Strings and integers are useful introductory key choices; key suitability depends on hashability. Values can themselves be records or collections.

Expression	Interpretation
`record[key]`	Retrieve a value; a missing key raises `KeyError`.
`record.get(key)`	Retrieve a value or return `None` when absent.
`record.get(key, default)`	Use an explicitly chosen missing-key result.
`key in record`	Ask about keys, not values.
`record[key] = value`	Insert an association or replace its existing value.
`record.items()`	Iterate over associated key–value pairs.
`record.pop(key)`	Remove an association and return its value.

Membership in keys is different from membership in a list of dictionary values. Likewise, a stored zero is not absence. Choose a missing-data policy that preserves that distinction.

6.6. Nested lookup must be checked at the intended level

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 86.

The material table connects a name to a property record. The property record then connects a property key to a numerical value. The two lookups represent different questions: whether the material exists and whether the requested property exists within that material.

```
# These constant densities are illustrative course values, not standards.
materials = {
    "steel": {"density_kg_m3": 7850},
    "aluminum": {"density_kg_m3": 2700}
}
selected = "steel"
density = materials[selected]["density_kg_m3"]  # Two keyed lookups.
print(density)
print(materials.get("titanium"))  # Missing material; do not invent zero.
```

Code 6.5: Look up illustrative density values without replacing an absent material with zero.

```
7850
None
```

Steel and aluminum retain the review's illustrative density values of 7850 and 2700 kg/m^3. An absent titanium entry is not a zero-density material. These values are programming examples, not certified design data.

6.7. Practice: attempt before consulting the solutions

6.7.1. M06-Q1: Before or after the recursive call?

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 88.

Predict the output order. What becomes smaller? What changes if `print(n)` is placed above the recursive call?

```
# The call receives a nonnegative integer; the zero case returns directly.
def show(n):
    if n == 0:
        return
    show(n - 1)  # Wait for the smaller call to finish first.
    print(n)  # This statement executes during the return phase.

show(3)
```

Code 6.6: Practice M06-Q1. Predict before running.

Worked solution: page 34

6.7.2. M06-Q2: Choose a missing-value policy

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 90.

Predict every printed line. Explain key membership versus value membership. Does `get` insert a missing entry?

```
readings = {"P-01": 0.0, "P-02": 120.0}
# Membership checks keys, not the numerical values.
print("P-01" in readings, 120.0 in readings)
print(readings.get("P-01"))
print(readings.get("P-03"))  # None is the chosen absence marker.
print("P-03" in readings)  # get() did not insert a new record.
```

Code 6.7: Practice M06-Q2. Predict before running.

Worked solution: page 35

6.7.3. M06-Q3: A recurrence is not a loop

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 92.

For $R(0) = 0$ and $R(n) = n + R(n-1)$ for positive integer n, expand $R(3)$, state its meaning, and compare recursion with a running total.

Worked solution: page 35

6.8. Worked solutions and reasoning

M06-Q1: worked solution

```
1
2
3
```

Calls descend through 3, 2, 1, and 0 before the first display occurs. The zero call returns directly. The waiting calls then print 1, 2, and 3 in that order during unwinding. Moving the display above the recursive call prints 3, 2, and 1 during descent. The termination argument applies to a nonnegative integer that decreases by one; a negative starting value is outside that argument.

Takeaway

Call order and the order of actions during return can be opposite.

Return to question M06-Q1 (page 33).

M06-Q2: worked solution

```
True False
0.0
None
False
```

The key `"P-01"` exists, but 120.0 is a value rather than a key in this dictionary. Looking up the first key returns its legitimate zero. Looking up `"P-03"` with `get` returns the default `None`; the key remains absent. Replacing absence with 0.0 would erase the distinction between no record and an actual zero reading.

Takeaway

Choose a missing-value marker that cannot be confused with an ordinary measurement.

Return to question M06-Q2 (page 34).

M06-Q3: worked solution

$R(3) = 3 + (2 + (1 + 0)) = 6$. The definition sums the integers from 1 through n. Both straightforward versions perform n additions under the stated unit-cost model. The recursive version additionally retains waiting calls; the loop uses one running subtotal and a loop state instead. The valid domain must exclude negative inputs, because decrementing a negative value does not approach zero.

Takeaway

Similar arithmetic work does not imply similar call-stack storage.

Return to question M06-Q3 (page 34).

6.9. Recursion and dictionary checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 93.

For recursion, identify the base case, progress measure, and returned combination. For a dictionary, identify the key convention, value schema, and absence policy. In both settings, a program can appear to run successfully while returning a result with the wrong meaning; use contracts and tiny traces to expose that problem.

7 Testing, Debugging, and Explicit Failure

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 94–108.

After reviewing this module

Choose tests from both contracts and execution paths, locate the first incorrect state, separate exception handling from success, and verify returned results, units, and data preservation together.

7.1. Correct with respect to a specification

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 95.

Testing compares an observation with an independently justified expectation. Debugging investigates a reproducible mismatch, finds its cause, repairs it, and repeats the tests. These activities are related, but they answer different questions.

Before testing, state the input types and units, valid ranges, required output, accuracy requirement, mutation policy, and failure behavior. A function can correctly implement a model that is unsuitable for a physical system. Conversely, an appropriate physical model can be implemented incorrectly. The evidence for implementation and the evidence for model suitability must not be conflated.

7.2. Choose tests for several reasons

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 96.

A known-answer test uses a result available independently, such as 10 N divided by 2 m^2 giving 5 Pa. A typical case exercises intended ordinary use. A boundary case examines equality and nearby values. An invalid-input case checks the documented rejection. A non-mutation test checks that a supposedly read-only operation leaves its input unchanged. A regression test preserves a case that previously exposed a defect.

These labels can overlap. A test at an inclusive pressure limit can be a boundary test, a unit test for one classifier, and a regression test after repairing a strict-inequality mistake. Integration tests additionally examine whether individually sensible components exchange compatible quantities.

Test purpose	Example input	Independent expectation
Known value	10,000 N; 500 mm^2	20 MPa.
Inclusive lower boundary	20°C for [20,30]	Normal.
Inclusive upper boundary	30°C for [20,30]	Normal.
Invalid area	Area zero	Documented rejection, not a stress of zero.
Non-mutation	Saved raw list and processed output	Raw list still equals its saved contents.

7.3. Combine black-box and glass-box reasoning

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 97.

Black-box cases come from the contract without relying on the implementation. If both temperature endpoints are normal, both must be tested even before the function exists. Glass-box reasoning inspects the implementation and asks which branches, returns, and loop exits have actually executed.

For a loop, test zero, one, and several passes; verify the last required element. For recursion, test each base case and at least one call requiring a recursive return. Executing every branch does not prove that its comparison or returned value is correct. Equally, ordinary contract examples may miss a rarely used failure route. Use both views.

7.4. Classify the symptom and read the traceback

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 98.

A syntax error prevents parsing. A runtime error raises an exception during execution. A logic error produces behavior that disagrees with the contract while the program still runs. Unit and data errors often produce plausible numerical answers with incorrect meanings. Numerical errors concern approximation, rounding, convergence, or accuracy requirements.

For a traceback, begin with the exception type and message, identify the failing expression, and follow the calls that supplied its inputs. The location at which failure is detected is not always where the incorrect value originated. Inspect values, types, and units along that path. `repr(text)` can expose otherwise invisible spaces, tabs, and line endings.

7.5. Repair one cause and keep the evidence

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 99.

A controlled investigation starts by reproducing the symptom with the same input. Reduce the example until the essential failure remains, then find the first intermediate state that differs from the expectation. Form a concrete hypothesis and change one cause rather than several unrelated lines.

After the repair, rerun the failing case and the existing tests. Record the input, intended result, observation, cause, repair, and retained regression test. Random edits that eventually produce a desirable final number do not explain why the code is now correct.

7.6. Exceptions make failure a separate path

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 100.

A calculation function can raise an exception when its input violates a contract. A caller can catch an expected exception and explain that no result was calculated. The exception is not itself a replacement numerical value.

```python
# This introductory function assumes finite numeric arguments.
def stress_pa(force_n, area_m2):
    if area_m2 <= 0:
        raise ValueError("area must be positive")
    return force_n / area_m2

try:
    result = stress_pa(10.0, 0.0)
except ValueError as error:
    print("Rejected:", error)  # Explain failure without a fake reading.
else:
    print("Stress:", result, "Pa")  # Only a successful try reaches here.
```

Code 7.1: The rejected calculation does not enter the success route.

```
Rejected: area must be positive
```

In a `try/except/else` statement, the successful `else` route is reached only when the protected block finishes without an exception. Code after the complete structure may still run. Catch the failure types that the interface expects; an overly broad handler can conceal unrelated programming defects.

An invalid proposed value and a valid but undesirable measurement are different. A finite reading outside an operating interval may need to be stored and flagged. A malformed input may instead need rejection. The decision belongs to the data contract, not merely to whether the number looks large.

7.7. Assertions are checks, not permanent input protection

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 101.

An assertion records a condition that should hold at a particular point. It is useful for known-answer tests, invariants, accounting identities, and regression evidence. Python can disable assertions under optimized execution; input validation that must always apply belongs in explicit checks and exceptions.

```python
# Validate types and physical domain before returning a number.
def stress_mpa(force_n, area_mm2):
    """Return uniform axial stress in MPa; reject invalid numeric input."""
    force_n = finite_number(force_n, "force_n")
    area_mm2 = finite_number(area_mm2, "area_mm2")
    if area_mm2 <= 0:
        raise ValueError("area_mm2 must be positive")
    # 1 N/mm^2 equals 1 MPa; a signed force keeps its sign.
    result = force_n / area_mm2
    if not math.isfinite(result):
        raise ValueError("calculated stress is not finite")
    return result
```

Code 7.2: The review's explicit numeric and physical-domain checks for uniform axial stress.

This function uses the identity $1\ \mathrm{N/mm^2} = 1\ \mathrm{MPa}$. It allows signed force under its contract but requires positive area. Its numeric helper rejects Boolean, nonnumeric, and nonfinite input;

those helper details are printed in Chapter 14.

```python
from review_models import stress_mpa
# Assertions check results; explicit validation remains in the function.
assert stress_mpa(10_000, 500) == 20.0
assert stress_mpa(0, 500) == 0.0
assert abs((0.1 + 0.2) - 0.3) < 1e-12  # Arithmetic criterion only.
print("Selected assertions passed")  # Added confirmation for the driver.
```

Code 7.3: Known-answer and boundary checks of a reusable calculation.

```
Selected assertions passed
```

An absolute tolerance has the same unit as the compared quantity. Its purpose must be stated: an arithmetic comparison tolerance is not the sensor's measurement uncertainty and does not validate a physical model.

7.8. Check the state as well as the result

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 102.

An operation may return the expected number while corrupting raw data or altering a second aliased object. Preserve a copy suitable for the data structure, call the operation, and inspect the promised post-state. For a rejected setter, test that the old valid value remains available.

Engineering trends are useful supplementary tests: doubling force should double the uniform axial stress, and doubling positive area should halve it. However, a constant wrong unit multiplier can satisfy both trends. Retain an independent absolute-value test as well.

7.9. Practice: attempt before consulting the solutions

7.9.1. M07-Q1: Two unit defects survive execution

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 103.

A script uses `area_m2 = 500 / 1000`, divides 10,000 N by that area, and then divides strength 250 MPa by the stress in Pa. Find both errors and give the correct stress and factor of safety.

Worked solution: page 40

7.9.2. M07-Q2: Trace exception control flow

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 105.

Write the displayed lines in order. Does the second statement in `try` run? Why is `else` skipped?

```python
try:
    value = float("bad")  # This conversion raises ValueError.
    print("converted")  # Skipped after the exception.
except ValueError:
    print("rejected")
else:
    print("success")  # Also skipped because try did not succeed.
print("finished")  # Ordinary execution resumes after the handler.
```

Code 7.4: Practice M07-Q2. Predict before running.

Worked solution: page 40

7.9.3. M07-Q3: Design the tests before the function

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 107.

A function returns the mean of a nonempty finite numeric list, rejects an empty input, and does not change the list. Propose four known-answer cases, an invalid case, and a state check.

Worked solution: page 40

7.10. Worked solutions and reasoning

M07-Q1: worked solution

The length conversion 10^{-3} must be squared for area: 500 $mm^2 = 500 \times 10^{-6} = 0.0005$ m^2. The correct stress is 10,000/0.0005 = 20,000,000 Pa, or 20 MPa. Strength and stress must use the same unit in their ratio, so FoS = 250/20 = 12.5. Python can perform every arithmetic operation in the faulty script; the defect concerns the interpretation of numbers and units, not syntax.

Takeaway

Unit tests require independent units as well as independent numerical expectations.

Return to question M07-Q1 (page 39).

M07-Q2: worked solution

```
rejected
finished
```

Converting `"bad"` raises `ValueError`. The following display in `try` is skipped, and the matching handler prints `rejected`. The `else` block is reserved for successful completion of the `try` block, so it is skipped. Execution resumes after the structure and prints `finished`. A handled error does not imply that a replacement numeric value exists.

Takeaway

Follow only the selected exception route, and do not read a result that was never assigned.

Return to question M07-Q2 (page 39).

M07-Q3: worked solution

Four independent cases are `[2,4]` → 3, `[5]` → 5, `[3,3,3]` → 3, and `[-2,2]` → 0. They exercise an ordinary case, a singleton, repeated values, and cancellation of signs. An empty list must trigger the documented rejection rather than division by zero or a fabricated mean. Save a copy before the call and compare the input afterward. Test nonfinite inputs when the implementation promises to reject them; do not invent an unstated exception type.

Takeaway

A complete test plan checks the promised outputs, failures, and side effects.

Return to question M07-Q3 (page 40).

7.11. Reliability checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 108.

Do not stop at “it runs.” Explain the contract, one ordinary case, one boundary case, one failure route, and the state that remains afterward. A passing collection of selected tests supports those observations, not a universal proof over every possible input or physical condition.

8 Classes, Instances, and Methods

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 109–119.

After reviewing this module

Distinguish a class from an instance, read receiver binding correctly, separate a query from an update, and trace object identity independently of equal stored values.

8.1. Organize one meaningful entity

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 110.

A class defines a type and its operations. An instance is one particular object of that type. Data attributes hold information associated with the instance; methods define related behavior. State means the information currently stored, whereas behavior describes what an operation can calculate, answer, or change.

A sensor is a useful introductory example because a name, unit, current reading, and interpretation rule naturally belong together. This does not make every dictionary inadequate. A small record with a few external functions can remain clearer than a class. Choose the representation by its responsibility, not by a desire to use the newest syntax.

8.2. Initialization establishes instance data

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 111–113.

Calling a class creates an instance and initializes its required fields. The conventional first parameter `self` identifies the receiver of an ordinary instance method. In `self.value = value`, the right-hand name is the local parameter; the left-hand expression assigns an attribute on the selected object.

```python
# This introductory class assumes valid readings and ordered limits.
"""Separate Week 8 teaching class, with caller-supplied valid finite data."""
class IntroSensor:
    """Store one reading, a unit, and inclusive limits; no validation here."""
    def __init__(self, name, unit, value, lower, upper):
        # Each instance receives its own state from the arguments.
        self.name = name
        self.unit = unit
        self.value = value
        self.lower = lower
        self.upper = upper

    def is_safe(self):
        """Return only compliance with the stated inclusive limits."""
        return self.lower <= self.value <= self.upper

    def update(self, new_value):
        # Caller guarantees valid numeric input; return is None.
        self.value = new_value

    def report(self):
        """Return labelled text; the caller decides where to display it."""
        status = "WITHIN LIMITS" if self.is_safe() else "OUTSIDE LIMITS"
        return f"{self.name}: {self.value:.1f} {self.unit} | {status}"
```

Code 8.1: The complete introductory class used by the self-study questions.

This introductory model assumes valid finite data and ordered limits. Its public attributes can be assigned directly. It is intentionally distinct from the explicitly validated `Sensor` class in Chapter 9; students should not silently combine the two definitions or attribute names.

The initializer stores data and does not return a replacement object. A local assignment without `self.`, such as `value = initial_value`, does not establish the intended instance attribute. Reading that missing attribute later raises an error even though the initializer itself may have run.

8.3. The dot chooses an object; parentheses perform the call

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 112–113.

For an ordinary method, `s.report()` supplies `s` as the receiver. In this example it has the same effect as `IntroSensor.report(s)`. The shorter instance form is the usual call syntax. This relationship explains why the explicit call supplies no separate `self` argument.

Expression	What is obtained or performed
`s.value`	Read the stored attribute.
`s.is_safe`	Obtain a bound method object, without running its body.
`s.is_safe()`	Run the query and obtain its Boolean answer.
`s.update(x)`	Change the current reading; return `None`.
`s.report()`	Obtain a formatted string using the current data.

In the review's classroom model, `is_safe()` means only that the reading lies within the supplied inclusive interval. The method name is retained from the source, but its result is not a physical-system safety certificate.

A saved Boolean result is a value obtained at the time of the call, not a live rule that will reevaluate itself whenever the reading changes. Call the method again to inspect a new state. Similarly, printing a report string and returning it are different interface decisions.

8.4. Equal data does not mean shared identity

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 114.

Two calls to the class create two distinct instances even when their initial fields are equal. Assigning a second name to one instance creates an alias. Reading a numeric attribute into a local variable obtains its current numeric object; later rebinding that local variable does not modify the instance attribute.

```
from intro_sensor import IntroSensor
# Added concrete driver for the source's identity discussion.
a = IntroSensor("A", "deg C", 25.0, 20.0, 30.0)
b = IntroSensor("B", "deg C", 25.0, 20.0, 30.0)
saved = a.value
a.update(35.0)  # Changes a, not b or the previously saved float.
print(a.value, b.value, saved)
```

Code 8.2: Worked driver for separate instances, an alias, and a saved numeric value.

```
35.0 25.0 25.0
```

Track which target appears on the left side of each assignment. An assignment to `a.value` changes a field of the object selected by `a`; an assignment to a plain local name changes that binding. An update through an alias affects the one shared instance, not every instance of the class.

Common mistake

Do not write `sensor = sensor.update(x)` when the method mutates in place and returns `None`. The update first changes the object; the subsequent assignment then replaces the local reference with `None`. Other aliases may still refer to the changed object.

8.5. Practice: attempt before consulting the solutions

8.5.1. M08-Q1: Saved query or live condition?

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 115.

Predict the three results. Why does the saved Boolean not change? Does `update` return the sensor?

```
from intro_sensor import IntroSensor
s = IntroSensor("Tank", "deg C", 25.0, 20.0, 30.0)
old_ok = s.is_safe()  # Evaluate the query at the original reading.
result = s.update(35.0)  # Change state; return None.
print(old_ok, s.is_safe(), result)  # The new call uses current state.
```

Code 8.3: Practice M08-Q1. Predict before running.

Worked solution: page 45

8.5.2. M08-Q2: Missing parentheses change the question

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 117.

Explain why `if s.is_safe:` does not query the method body. Then explain why `s = s.update(35)` loses the sensor reference.

Worked solution: page 45

8.6. Worked solutions and reasoning

M08-Q1: worked solution

```
True False None
```

The original reading 25.0 lies in the inclusive 20–30 interval, so the saved query result is `True`. Updating the same sensor to 35.0 changes the current query to `False`. It does not rerun the expression that produced `old_ok`. The update has no explicit return value, so its result is `None`, not a replacement sensor.

Takeaway

Saved results describe the state when they were computed; a new query is needed for new state.

Return to question M08-Q1 (page 44).

M08-Q2: worked solution

Without parentheses, `s.is_safe` is a bound method object, which is truthy in this ordinary example. Its body has not evaluated the limits. Use `s.is_safe()` to obtain the Boolean result. The update method changes the sensor but returns `None`. Assigning that return value back to `s` therefore replaces the name's reference with `None`; call `s.update(35)` as a separate statement instead.

Takeaway

Reading a method, calling it, and assigning its returned value are three different operations.

Return to question M08-Q2 (page 45).

8.7. Object checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 119.

For every method call, identify the receiver, the attributes read, any attributes changed, and the object returned. Test a second independent instance, not only one successful call. That discipline connects OOP to the same contracts, references, and tests already used for functions and collections.

9 Interfaces, Shared State, and Inheritance

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 120–133.

After reviewing this module

Preserve an invariant through supported updates, trace class and instance attributes, recognize shared mutable state, and explain inheritance as compatible specialization rather than merely reused syntax.

9.1. Validate before committing a state change

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 121.

A getter provides a documented quantity without changing the object. A setter accepts a proposed replacement, checks it, and only then assigns the stored attribute. If validation raises before the assignment, the old state remains intact.

The reviewed sensor class accepts finite numeric readings and ordered finite limits. It rejects Boolean and nonnumeric values according to its explicit helper contract. A valid reading outside the operating interval is still retained; it is classified as outside the limits rather than treated as nonexistent data. A Celsius temperature is not constrained by the nonnegative absolute-pressure rule used in a different course example.

```python
# The validated model is separate from the simpler IntroSensor.
class Sensor:
    """One finite reading with ordered limits; units are documented text."""
    display_digits = 1  # A class-level reporting default, not a reading.

    def __init__(self, name, unit, value, lower, upper):
        self.name, self.unit = name, unit
        self.lower = finite_number(lower, "lower")
        self.upper = finite_number(upper, "upper")
        if self.lower > self.upper:
            raise ValueError("limits must be ordered")
        self.set_value(value)  # Establish a valid initial reading.
# END sensor_init

    # BEGIN sensor_methods
    def get_value(self):
        """Return the current numeric reading."""
        return self._value

    def set_value(self, proposed):
        """Reject invalid input before replacing existing state."""
        checked = finite_number(proposed, "reading")
        self._value = checked  # Valid but out-of-limit data are retained.

    def within_limits(self):
        """Return compliance with this object's inclusive limits."""
        return self.lower <= self._value <= self.upper
    # END sensor_methods

    def report(self):
        """Return a string without mutating the stored reading."""
        status = "WITHIN LIMITS" if self.within_limits() else "OUTSIDE LIMITS"
        return (f"{self.name}: {self._value:.{self.display_digits}f} "
                f"{self.unit} | {status}")
```

Code 9.1: The validated sensor class used by the review, separate from the introductory model.

The helper `finite_number` is printed in Chapter 14 and supplied in the accompanying module. Test both successful replacement and failed replacement. Checking only that an exception occurs does not prove that the previous reading survived the failure.

9.2. Hide representation, not units and responsibilities

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 122.

A public interface allows client code to depend on documented operations rather than backing attribute names. A single leading underscore communicates that an attribute is intended as an internal implementation detail. It is a convention for cooperation, not a security boundary.

A getter returning an internal list may expose mutable data even though the attribute name has an underscore. A copying getter can protect the outer list when a snapshot is promised. For nested mutable data, decide whether sharing inner objects is permitted; an outer copy does not establish deep independence. Information hiding therefore includes control of references, not merely a spelling convention.

9.3. Class defaults and instance overrides

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 123.

An ordinary class attribute can provide a shared default, such as the number of decimal places in a report. Assigning a value to `instance.display_digits` creates or updates an attribute on that instance. It does not necessarily change the class default or a different object's override.

For the ordinary attributes in this review, read an instance's own value first if present; otherwise use the inherited class default. An instance override can therefore shield that instance from a later change to the class setting. This is the simple lookup model required by the exercise, not a complete description of every advanced Python descriptor mechanism.

9.4. A mutable class attribute can share more than intended

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 124.

An experiment history normally belongs to one experiment. If its empty list is created in the class body, every instance that uses that class attribute initially refers to the same list. Appending through one instance changes the shared object. Placing `self.history = []` in the initializer instead creates a new list for each instance.

```
# First class deliberately couples what should be separate histories.
class BadLog:
    history = []
    def add(self, value):
        self.history.append(value)  # Mutate the class-level list.

# Repair: allocate a new list for each successfully created instance.
class GoodLog:
    def __init__(self):
        self.history = []
    def add(self, value):
        self.history.append(value)

# Added demonstration of the difference, using fresh class definitions.
a, b = BadLog(), BadLog()
a.add(1)
print("Shared:", b.history)
c, d = GoodLog(), GoodLog()
c.add(1)
print("Independent:", d.history)
```

Code 9.2: The deliberately shared-list model and its per-instance repair.

```
Shared: [1]
Independent: []
```

The contrast is between mutation and rebinding. Appending to an inherited list changes that one list. Assigning a new list to an instance attribute establishes a different reference for that instance. Do not repair the defect by changing the expected test result: repair the ownership of the state.

9.5. Specialize a class without breaking its contract

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 125–127.

Inheritance expresses an "is-a" relationship: a temperature sensor is a specialized sensor. A child can reuse initialization and behavior, add operations, or override an operation while preserving its expected meaning and return type.

```python
# Inherited methods still operate on this particular child instance.
class TemperatureSensor(Sensor):
    """Specialize the unit and report; retain the parent's update rule."""
    def __init__(self, name, value, lower=20.0, upper=30.0):
        super().__init__(name, "deg C", value, lower, upper)

    def report(self):
        # Reuse the parent report and add a type-specific prefix.
        return "Temperature | " + super().report()
```

Code 9.3: A temperature sensor uses parent initialization and extends reporting.

The `super()` call continues method lookup to inherited behavior for the same receiver. It does not create a separate parent object. Parent initialization assigns attributes on the child instance. When the child report obtains the parent's report string and adds a prefix, both parts describe the same stored sensor. A child that overrides a report by printing instead of returning text

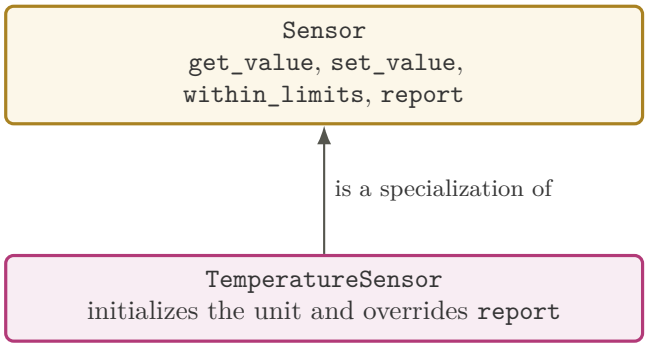


Figure 9.1.: The review's specialization relationship. Inherited operations still act on the actual child instance. A collection owned by an experiment would instead be a "has-a" relationship.

breaks clients that expect a string. Reusing the method name is not enough. Similarly, a query should not introduce an unexpected mutation, and a rejection should preserve any state promised by the parent contract.

Missing parent initialization can leave inherited operations without the attributes they expect. Test a successfully initialized child, an inherited boundary, a rejected update, and a separate instance. Use composition when one entity contains or uses other entities rather than being a specialized version of them.

9.6. Practice: attempt before consulting the solutions

9.6.1. M09-Q1: Which precision value is found?

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 128.

Predict the three integers. Which instance has an override? Why does changing the class setting affect `b`?

```
class Display:
    digits = 1  # Shared default in this simple ordinary-attribute model.

a = Display()
b = Display()
a.digits = 3  # Create an attribute on a only.
Display.digits = 2  # Change the fallback used by b.
print(a.digits, b.digits, Display.digits)
```

Code 9.4: Practice M09-Q1. Predict before running.

Worked solution: page 51

9.6.2. M09-Q2: Does rejection preserve state?

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 130.

Predict both printed lines. Which step prevents overwriting the old value? Is 35°C valid but outside the limits for this model?

```
from review_models import TemperatureSensor
sensor = TemperatureSensor("Tank", 25.0)
try:
    sensor.set_value(float("nan"))  # Nonfinite proposal is invalid.
except ValueError:
    print("rejected")
print(sensor.get_value())  # The old finite reading must survive.
```

Code 9.5: Practice M09-Q2. Predict before running.

Worked solution: page 51

9.6.3. M09-Q3: Parent initialization was skipped

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 132.

A child defines an initializer but does not call the parent initializer that establishes its name, limits, and value. An inherited report reads those attributes. What failure is likely, and what should be tested after repair?

Worked solution: page 51

9.7. Worked solutions and reasoning

M09-Q1: worked solution

```
3 2 2
```

Instance `a` has its own `digits = 3`; this ordinary attribute shadows the class setting during lookup through `a`. Instance `b` has no override, so its lookup finds the changed class value 2. The class itself also stores 2. Changing a shared default does not erase a pre-existing instance override.

Takeaway

Identify both the object storing an attribute and the path used to look it up.

Return to question M09-Q1 (page 50).

M09-Q2: worked solution

```
rejected
25.0
```

The finite-number helper rejects NaN with `ValueError` before the assignment to `self._value`. The handler prints `rejected`; the getter then returns the unchanged 25.0. A finite 35.0 proposal is valid for this generic Celsius reading model and is stored, while the inclusive limit query returns `False`. Invalid input and out-of-limit valid evidence are not the same category.

Takeaway

Test the stored state after failure, not just that an exception was raised.

Return to question M09-Q2 (page 50).

M09-Q3: worked solution

An `AttributeError` is likely when the inherited code reads a missing attribute. Call the appropriate parent initializer with the necessary arguments, or deliberately establish the same required state and contract. After repair, test initialization, inherited boundary behavior, valid updates, rejection with state preservation, and independence of two instances. A report that runs once is not enough to check those obligations.

Takeaway

Inheriting a method does not automatically run a parent initializer that a child overrides.

Return to question M09-Q3 (page 50).

9.8. Interface checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 133.

State the public unit convention, identify who owns each mutable object, and explain what remains true after each supported operation. Changes to representation or inheritance should be accompanied by the same public-behavior tests, not by assumptions that syntactically valid reuse is automatically compatible.

10 Efficiency: Counts and Orders of Growth

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 134–147.

After reviewing this module

Define input size and a cost model, separate an input case from a bound notation, derive selected-operation counts, and distinguish evidence from timing experiments from general growth arguments.

10.1. A complete claim begins with the task

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 135.

An efficiency statement needs more than a label such as "linear." Define what n represents, the output required, the input case under consideration, and the operations being counted. State whether the model treats numeric operations and comparisons as constant-size work and whether input construction, output storage, or preprocessing is included.

The same traversal can be quadratic in the number of points on one side of a square grid but linear in the total number of grid points. Likewise, a method that returns the first warning and one that counts all warnings solve different tasks; early termination can be correct for the former and incorrect for the latter.

10.2. Count every relevant operation, not only interesting events

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 136.

A warning event and a threshold comparison are not the same quantity. A full scan of n readings compares every reading with the limit even when none produces a warning.

```python
# Caller supplies finite readings and a compatible limit.
def count_warnings(readings, limit):
    count = 0
    for value in readings:
        if value > limit:  # Executed once per reading, including normal ones.
            count += 1
    return count

print(count_warnings([19.0, 30.0, 31.0, 35.0], 30.0))
print(count_warnings([], 30.0))  # Added empty-input demonstration.
```

Code 10.1: Count selected comparisons separately from warning events.

```
2
0
```

For an empty list, the selected comparison count is zero, but function entry, initialization, and returning still require work. Exact selected-operation counts deliberately omit other operations; they are models used to explain growth, not literal counts of machine instructions.

10.3. Big-O is an eventual upper bound

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 137.

For a nonnegative work function $T(n)$, writing $T(n) = O(g(n))$ means that positive constants c and n_0 exist such that

$$T(n) \leq c\, g(n) \qquad \text{for all } n \geq n_0.$$

For example, $3n + 20 \leq 4n$ whenever $n \geq 20$, which establishes an $O(n)$ upper bound for that count. It also has a matching lower linear bound, so $\Theta(n)$ is a tight growth description. A looser $O(n^2)$ statement can be mathematically true but less informative.

The notation does not select the input case. Analyze the best-case function, worst-case function, or an average defined over a stated distribution, and then bound that particular function. “Big-O means worst case” is therefore not an adequate definition.

10.4. Dominant terms and finite-size decisions

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 138.

For $C(n) = 1000 + n + 2n^2$, the quadratic term eventually dominates and the count has tight quadratic growth. Ignoring lower-order terms for an asymptotic classification does not remove those operations from an actual execution.

Consider the declared models $A(n) = 100n$ and $B(n) = n^2$. They agree at $n = 100$; below that value the quadratic expression is smaller, and above it the linear expression is smaller. These are model work units, not measured milliseconds. A better large-input growth rate does not by itself identify the faster implementation at every finite size.

10.5. Best, worst, and assumed average cases

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 139.

A first-match linear scan on a nonempty list needs one selected comparison if the first item matches. A match only at the last position or an absent target requires n. If a successful first-match position is assumed uniformly distributed among the n positions, the conditional average is

$$\frac{1 + 2 + \cdots + n}{n} = \frac{n + 1}{2}.$$

That average is not automatically appropriate to a real sensor log, where absence, clustered events, or a nonuniform position distribution may matter. State the assumption rather than treating it as a universal property of linear search.

10.6. Recognize the structure behind each growth class

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 140–141,143.

A fixed number of accesses gives constant work. Repeated halving gives a logarithmic number of reductions. A full scan gives linear work. Work that scans all data across a logarithmic number of levels often gives $n \log n$. Independent nested dimensions multiply; a triangular range must be summed rather than simply counted as a square.

Model	Characteristic pattern	Effect of doubling n
1	A fixed number of operations.	No growth in this count.
$\log_2 n$	Halve the remaining problem.	Add one level.
n	Visit each input once.	Multiply by two.
$n \log_2 n$	Linear work across logarithmic levels.	Slightly more than double.
n^2	All ordered pairs.	Multiply by four.
n^3	Three independent size-n ranges.	Multiply by eight.
2^n	All subsets or binary choices.	Square the value, a much steeper change.

Consecutive blocks add their costs. For nested loops, determine how often the inner work actually executes: a fixed five-pass inner loop gives $5n$, whereas an inner range of length n gives n^2. With inner lengths $0, 1, \ldots, n-1$, the count is $n(n-1)/2$.

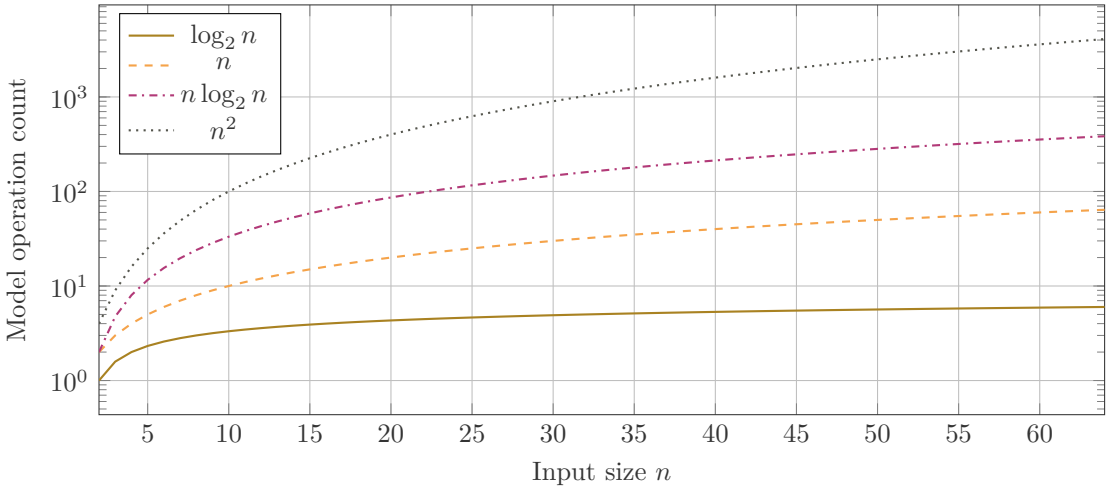


Figure 10.1.: Recreated model growth curves from the review. The vertical scale is logarithmic; these mathematical counts are not elapsed-time measurements.

10.7. Measure a precisely defined region

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 142.

An elapsed duration is the difference between two counter readings. Build inputs before the timer only when the stated comparison is calculation-only. Put printing, plotting, and correctness checks outside that region, while remembering that those operations still contribute to an end-to-end application.

```python
from time import perf_counter
# Construct the data before starting this explicitly limited measurement.
values = list(range(10_000))
start = perf_counter()
total = sum(values)
elapsed_s = perf_counter() - start
assert total == 49_995_000  # Independent arithmetic answer, after timing.
print(elapsed_s)  # This duration is machine- and run-dependent.
```

Code 10.2: A timing demonstration with a hand-checkable result.

The deterministic sum is $0 + 1 + \cdots + 9999 = 49{,}995{,}000$. The duration varies with hardware, implementation, input, and background activity, so no fixed elapsed output is supplied. A fair comparison uses equivalent inputs, the same required result, several sizes, and repeated trials. Keep raw measurements and state the chosen summary. A timing trend supports an interpretation over tested conditions; it does not replace the algorithmic argument.

10.8. Practice: attempt before consulting the solutions

10.8.1. M10-Q1: A fixed inner bound

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 144.

Predict the count. Express it for general nonnegative n. Give its tight growth under the selected unit-cost model.

```python
# Count selected inner-loop visits, not elapsed seconds.
n = 4
visits = 0
for i in range(n):
    for j in range(5):  # Five passes independent of n.
        visits += 1
print(visits)
```

Code 10.3: Practice M10-Q1. Predict before running.

Worked solution: page 56

10.8.2. M10-Q2: An upper bound need not be tight

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 146.

For $T(n) = 3n + 20$, is $O(n^2)$ a valid upper bound? Is $\Theta(n^2)$ the correct tight growth? Explain.

Worked solution: page 56

10.9. Worked solutions and reasoning

M10-Q1: worked solution

```
20
```

The outer loop has n passes. Each contains exactly five selected visits, independent of n, so the count is $5n$. For $n = 4$, this gives 20. The tight asymptotic class is $\Theta(n)$, despite the two levels of indentation. For $n = 0$ the selected body count is zero, although program setup still takes work.

Takeaway

Nested syntax does not establish independent size-dependent repetitions.

Return to question M10-Q1 (page 55).

M10-Q2: worked solution

$O(n^2)$ is true but loose: for $n \geq 20$, $3n + 20 \leq 4n \leq 4n^2$. The same cost is also at least $3n$ for positive n, so its tight growth is $\Theta(n)$, not $\Theta(n^2)$. A loose bound can obscure the difference between a linear and a genuinely quadratic algorithm. The notation does not itself choose a best, worst, or average input case.

Takeaway

State the cost function first; distinguish a true upper bound from the most informative justified description.

Return to question M10-Q2 (page 55).

10.10. Efficiency checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 147.

Say what grows, why it grows, and what was excluded. Then ask whether the faster version preserves the result, units, mutation policy, and accuracy. An implementation that saves work by omitting required outputs has changed the task rather than optimized it.

11 Hidden Work, Recursion, and Memory

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 148–162.

After reviewing this module

Include work performed by library operations and called methods, distinguish total work from peak live storage, account for preprocessing and caching, and test whether an efficient numerical update preserves the required accuracy.

11.1. A short expression can hide a full scan

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 149.

Under the introductory Python-list model, an indexed access takes fixed work, but summing a list visits its elements. A slice copies the selected references. Removing the first item shifts the remaining references; removing the last does not require that shift. Appending has constant amortized cost over a sequence of operations, although an individual append can require a larger allocation and copying step.

Dictionary lookup is usually described by an expected constant-cost model under appropriate bounded-size hashing assumptions. This is not an unconditional worst-case guarantee for arbitrary keys and collisions. Code length is therefore a poor proxy for work: inspect the operation and its data representation.

11.2. Repeated construction can repeatedly copy the prefix

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 150.

If a growing output is rebuilt by concatenating a one-element list on every pass, the old prefix is copied repeatedly. Across n additions, the old-prefix lengths sum to $0 + 1 + \cdots + (n - 1)$, giving quadratic cumulative copying. Appending to the same list instead supports linear total construction under the amortized model.

```
values = [1, 2, 3]  # Added concrete input for the displayed patterns.
# Inefficient version: recreate the growing list each pass.
result = []
for value in values:
    result = result + [2 * value]
print(result)
# Same values and order, but reuse a single result list.
result = []
for value in values:
    result.append(2 * value)
print(result)
```

Code 11.1: Two constructions with the same required result but different hidden copying.

```
[2, 4, 6]
[2, 4, 6]
```

Both outputs have n elements. The cumulative amount of copying is not the same as the maximum number of references simultaneously alive. Some temporary lists can be released; other recursive patterns keep them alive. Always specify which storage measure is meant.

11.3. Preprocessing is part of the cost

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 151.

Checking each of n installed identifiers against a list of m approved identifiers can require nm equality tests in the worst case. Building a dictionary index once can change the expected total to $O(m + n)$ under the stated key-cost model, at the price of additional storage.

```python
# Retain one Boolean result per installed occurrence, including repeats.
installed = ["A", "B", "A", "C"]
approved = ["A", "C"]
index = {}
for name in approved:
    index[name] = True  # Build once; duplicates in approved collapse safely.
flags = []
for name in installed:
    flags.append(name in index)  # Preserve installed order and length.
print(flags)
```

Code 11.2: A worked installed-ID driver preserves the required order and repeated identifiers.

```
[True, False, True, True]
```

The driver values are an illustrative completion of the review's pattern. The output contains one Boolean for each installed occurrence, so both occurrences of A retain separate results in their original positions. Replacing this with a unique set of identifiers would change the task. If approval data change, rebuilding or updating the index also belongs to the application cost.

11.4. Recursive cost is local work times the number of calls

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 152–153.

A function that reduces n by one with constant local work has linear total work and potentially linear active depth. One recursive half with constant local work gives logarithmic work and depth. Two disjoint halves with a full linear combination at each level can give $\Theta(n \log n)$ total work even though the depth is logarithmic.

```python
# Small lists only: both versions use linear recursive depth.
def sum_index(values, index=0):
    if index == len(values):
        return 0
    # Reuse the original list; change only the position.
    return values[index] + sum_index(values, index + 1)

def sum_slice(values):
    if not values:
        return 0
    # Each tail slice creates a new outer list retained by a waiting call.
    return values[0] + sum_slice(values[1:])

print(sum_index([1, 2, 3]), sum_slice([1, 2, 3]))  # Added check.
```

Code 11.3: Index-based summation and summation over successively copied tails.

```
6 6
```

The index-based version refers to the same input throughout. The slicing version creates a new tail at each call. Tail lengths $n-1, n-2, \ldots, 1$ sum quadratically; because unfinished callers can still refer to earlier tails, many of those copied references remain live simultaneously. The shared mathematical answer does not make the memory behavior identical.

For Fibonacci, the total branching tree has exponentially many calls in the naive implementation, while its active depth grows linearly. Counting only the longest chain underestimates work; counting every call as simultaneously active overestimates depth.

11.5. A cache changes which subproblems are new

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 154.

With an explicit dictionary cache, a subproblem already solved can be retrieved rather than recursively recomputed. The review's Fibonacci convention requires cached base values 1 at keys 0 and 1. A fresh-cache computation introduces linearly many distinct states under the unit-cost arithmetic model. A later lookup of a stored result is a different task from computing it for the first time.

```python
# Caller supplies valid n and a compatible cache with correct entries.
def cached(n, cache):
    if n not in cache:
        # Compute a previously missing state once and retain its value.
        cache[n] = cached(n - 1, cache) + cached(n - 2, cache)
    return cache[n]

cache = {0: 1, 1: 1}  # Fresh cache for this demonstration.
print(cached(5, cache))
print(sorted(cache.items()))  # Added view of the retained states.
```

Code 11.4: A compact cache demonstration under a valid-input and correct-cache contract.

```
8
[(0, 1), (1, 1), (2, 2), (3, 3), (4, 5), (5, 8)]
```

The cache retains results and the recursion retains unfinished calls. Moreover, large Fibonacci integers occupy more than a fixed-size scalar. The classroom linear-state argument should not be silently promoted to an exact bit-operation or byte-storage claim.

11.6. Moving-window means: preserve every requested output

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 155.

For n readings and a full window width w with $1 \le w \le n$, there are $n - w + 1$ requested means. The direct implementation sums each window independently. The running implementation sums the first window, then removes the outgoing value and adds the incoming value.

```python
# Recompute each full window in its own source order.
def window_means_direct(values, width):
    """Return all complete-window means; assume finite moderate values."""
    if type(width) is not int or not 1 <= width <= len(values):
        raise ValueError("width must be an integer from 1 to len(values)")
    means = []
    for start in range(len(values) - width + 1):
        total = 0.0
        for index in range(start, start + width):
            total += values[index]  # Revisit each item in each window.
        means.append(total / width)
    return means
```

Code 11.5: Direct full-window means with an explicit width contract.

```python
# Reuse one subtotal; test roundoff separately from arithmetic counts.
def window_means_running(values, width):
    """Reuse a running subtotal; roundoff can differ from direct sums."""
    if type(width) is not int or not 1 <= width <= len(values):
        raise ValueError("width must be an integer from 1 to len(values)")
    total = sum(values[:width])  # Build the first window subtotal once.
    means = [total / width]
    for end in range(width, len(values)):
        total -= values[end - width]
        total += values[end]
        means.append(total / width)
    return means
```

Code 11.6: Running full-window means using a reused subtotal.

Counting only additions and subtractions in the selected subtotal updates gives

$$C_{\text{direct}} = w(n - w + 1), \qquad C_{\text{running}} = w + 2(n - w) = 2n - w.$$

These counts exclude division, validation, loop overhead, and output storage. If w is fixed, both models are linear in n. The direct model can be quadratic when w grows proportionally to n, but when $w = n$ it computes only one window and is linear. A statement involving both variables must retain their relationship.

11.7. Algebraic equivalence is not identical floating-point behavior

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 156.

The running update is algebraically correct, but rounding can remove information needed later. With `[1e16, 1.0, 1.0]` and width two, the small contribution can be lost in the first large subtotal. Removing the large term later cannot reconstruct the lost value.

```
from review_models import window_means_direct, window_means_running
# The second window [1.0, 1.0] has the exact arithmetic mean 1.0.
values = [1e16, 1.0, 1.0]
print(window_means_direct(values, 2))
print(window_means_running(values, 2))  # Earlier roundoff survives updates.
```

Code 11.7: The review's numerical counterexample to unchecked running updates.

```
[5000000000000000.0, 1.0]
[5000000000000000.0, 0.5]
```

The second exact mathematical window has mean 1.0. The direct implementation obtains 1.0 for that small window, whereas the running example gives 0.5. This does not prove that the direct approach is accurate for every input. It demonstrates why an independently checkable case and a stated tolerance are required before accepting a numerical optimization.

11.8. Move unchanged work outside a loop only when valid

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 159–160.

Recomputing the mean of an unchanged list for every output element introduces a full scan inside another full scan. Compute the unchanged quantity once and reuse it. This repair depends on the input and required mean being constant throughout the transformation.

```
# This driver supplies a nonempty finite, moderate-scale list.
readings = [20.0, 22.0, 24.0]
mean = sum(readings) / len(readings)  # One full scan before output creation.
centered = []
for reading in readings:
    centered.append(reading - mean)  # Reuse the same scalar result.
print(centered)
print(readings)  # Raw data remain available.
```

Code 11.8: Worked centering repair: one mean calculation, then one output pass.

```
[-2.0, 0.0, 2.0]
[20.0, 22.0, 24.0]
```

The supplied concrete driver is labelled as a worked completion. It preserves the centered outputs while removing repeated work; it does not change the meaning of the mean or mutate the original readings.

11.9. Practice: attempt before consulting the solutions

11.9.1. M11-Q1: A shrinking inner workload

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 157.

Predict the selected visit count, write the inner-loop lengths, and give the tight growth for a positive initial size n.

```
size = 8
visits = 0
while size > 0:
    for _ in range(size):
        visits += 1  # Count work at the current size.
    size //= 2  # Next inner loop has roughly half as many passes.
print(visits)
```

Code 11.9: Practice M11-Q1. Predict before running.

Worked solution: page 62

11.9.2. M11-Q2: A helper call hides a full scan

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 159.

A program subtracts the mean from each reading but recomputes `sum(readings)/len(readings)` inside the output loop. Classify the work and propose a contract-preserving repair.

Worked solution: page 63

11.9.3. M11-Q3: Operation count and output count differ

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 161.

For $n = 6$ and width $w = 3$, how many full-window means are needed? Calculate the selected arithmetic counts for direct and running-subtotal versions. Does fewer updates alone establish a better implementation?

Worked solution: page 63

11.10. Worked solutions and reasoning

M11-Q1: worked solution

```
15
```

The lengths are 8, 4, 2, and 1, giving $8+4+2+1 = 15$. In general the count is $n+\lfloor n/2\rfloor+\lfloor n/4\rfloor+\cdots$. It is at least n and less than $2n$, so tight growth is $\Theta(n)$. Multiplying n by the number of levels incorrectly treats every inner stage as having the original length.

Takeaway

Dependent bounds require a sum, not blind multiplication of maximum sizes.

Return to question M11-Q1 (page 62).

M11-Q2: worked solution

For n unchanged readings, each of n outer passes triggers a complete sum, giving quadratic work under bounded-cost arithmetic. Compute the mean once before the loop and build the centered result in one pass. The total work becomes linear under that model. Preserve the required order and raw data; handle an empty input according to the contract. Hoisting the sum is valid only while its inputs remain unchanged.

Takeaway

A short helper call can contain the dominant work of a program.

Return to question M11-Q2 (page 62).

M11-Q3: worked solution

The result must contain $n - w + 1 = 4$ means. Direct summation uses $w(n - w + 1) = 3 \times 4 = 12$ selected additions. The running-subtotal count is $w + 2(n - w) = 2n - w = 9$. These counts omit division, loop conditions, and output writes. Fewer selected updates do not establish faster execution or adequate accuracy. Check the same output sequence, numerical tolerance, memory, and realistic runtime conditions.

Takeaway

Output requirements, arithmetic counts, and measured cost are different quantities.

Return to question M11-Q3 (page 62).

11.11. Hidden-work checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 162.

Inspect calls, slices, copies, shifts, and setup. Distinguish cumulative work, retained output, auxiliary storage, and active depth. Verify numerical accuracy and the original output contract before treating a measured speedup as an engineering improvement.

12 Searching and Sorting

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 163–181.

After reviewing this module

Specify exactly what a search returns, justify discarding candidates in ordered data, distinguish insertion from membership, and sort complete records while preserving the promised multiplicity, tie policy, and input behavior.

12.1. The search question determines the algorithm

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 164.

"Find a value" can mean several different tasks. A membership test returns a Boolean. A first-match search returns the earliest matching index. An all-match search returns every qualifying record. A best-feasible search compares qualifying alternatives using an objective and tie policy. An insertion search returns where an ordered sequence could accept a value, whether or not the value already occurs.

State the input order, equality or tolerance rule, absence result, and mutation policy. Index zero must not be confused with failure. A function returning `None` for absence should be checked with an explicit `is not None`, rather than the truth value of the returned index.

12.2. Linear search preserves a checked-prefix invariant

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 165.

At the beginning of each pass, the earlier positions have been checked and none contains a match. If the current value matches, returning now is correct for a first-match contract. If the scan ends, every candidate has been checked and absence is justified.

```
# Examine positions in source order to preserve the first-match promise.
def linear_search(values, target):
    """Return the first matching index, or None; do not reorder values."""
    for index, value in enumerate(values):
        if value == target:
            return index  # Zero is a valid successful result.
    return None  # Absence is reported only after all candidates fail.
```

Code 12.1: Return the first matching index, or an explicit absence value.

For an empty list there are no candidates, so absence is immediate. For a nonempty list, a first-position match requires one equality test; a last-position match or missing target can require n. Duplicates do not invalidate the algorithm, but the return position is governed by its first-match policy.

12.3. Brute force is systematic candidate enumeration

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 166.

The review's resistor-pair pattern examines distinct unordered component pairs. The indices satisfy $i < j$, so an item is not paired with itself and the same unordered pair is not counted twice.

```python
# Distinct indices model distinct components, even for equal resistances.
def feasible_pairs(resistors, target):
    """List unordered index pairs whose series resistance equals target."""
    pairs = []
    for i in range(len(resistors)):
        for j in range(i + 1, len(resistors)):
            if resistors[i] + resistors[j] == target:
                pairs.append((i, j))  # Do not reuse the same component.
    return pairs
```

Code 12.2: Enumerate distinct unordered index pairs satisfying the target sum.

The candidate set contains $n(n-1)/2$ pairs. A returned pair of indices identifies actual positions, which matters if equal resistance values correspond to different available components. The exact equality in this teaching example is part of its input assumption. A tolerance-based measured-value problem would require a separately specified comparison rule.

First feasible, all feasible, and minimum-cost feasible are not interchangeable results. Stopping after one pair is appropriate only when one qualifying pair satisfies the stated task.

12.4. Bisection search depends on sortedness

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 167.

Numerical bisection in Chapter 3 searches for a root of a function. Binary search here searches an ascending stored sequence. Its evidence for discarding candidates comes from order, not continuity or residual signs.

```python
# The sorted-input precondition is assumed, not rechecked on each call.
def binary_search(values, target):
    """Return any matching index, or None; values must be ascending."""
    low, high = 0, len(values) - 1
    while low <= high:
        mid = (low + high) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            low = mid + 1  # Exclude the tested midpoint.
        else:
            high = mid - 1
    return None
```

Code 12.3: Inclusive-bound binary search under an ascending-order precondition.

The candidate interval is `low` through `high`, inclusive. If the midpoint is smaller than the target, every earlier candidate is too small and the next low value becomes `mid + 1`. If it is larger, the next high value becomes `mid - 1`. Each unsuccessful pass excludes the tested midpoint. When

low exceeds high, no candidates remain.

On unsorted data, the code may execute normally yet discard the position containing the target. Do not reinterpret a resulting `None` as reliable evidence of absence when the sortedness precondition was violated.

12.5. Trace both the values and the bounds

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 168.

For `[8, 12, 18, 22, 31, 40, 55]` and target 31, the complete successful path is:

Low	High	Mid	Value	Decision
0	6	3	22	Value too small: set low to 4.
4	6	5	40	Value too large: set high to 4.
4	4	4	31	Exact match: return index 4.

The final interval containing one candidate is still valid and must be checked. This explains why the condition is `low <= high`, not a strict comparison. For duplicates, this implementation may return any matching position encountered on its path; it does not promise the leftmost one.

12.6. Insertion positions are not evidence of membership

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 169.

The left insertion position partitions a sorted sequence into values below the target and values not below it. The position may equal the sequence length. To turn it into an exact-membership result, check that the position is in range and that the value there equals the target.

```
from bisect import bisect_left
values = [10, 20, 20, 40]  # Ascending input is a precondition.
target = 30
pos = bisect_left(values, target)
found = pos < len(values) and values[pos] == target
print(pos, found)  # Added output: an insertion position is not a match.
```

Code 12.4: An in-range insertion position can still point to a different value.

```
3 False
```

For the review sequence `[10, 20, 20, 40]`, target 30 belongs at position 3 but does not exist in the list. Target 20 has left insertion position 1. A sorted view made from a time series must also retain original indices or complete records when acquisition positions will be reported later.

12.7. Sorting has a contract beyond increasing numbers

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 170–171.

Sorting should preserve every required item, including duplicates, and maintain all fields belonging to each record. Specify ascending or descending order, the key, tie handling, and whether the original list can change.

```
raw = [3, 1, 2, 1]
ranked = sorted(raw)  # Return a new outer list; raw is unchanged so far.
result = raw.sort()  # Now mutate raw, and return None.
print(ranked, raw, result)  # Added output exposes all three results.
```

Code 12.5: A new sorted list versus an in-place sort and its return value.

```
[1, 1, 2, 3] [1, 1, 2, 3] None
```

```
# Teaching property values; the label must stay with its density.
records = [("steel", 7850), ("aluminum", 2700)]
def density_key(record):
    return record[1]  # Choose the field used for ordering, not a new record.

ranked = sorted(records, key=density_key)
print(ranked)
print(records[0][0])  # The input list still starts with steel.
```

Code 12.6: Sort complete material records using the property that defines the order.

```
[('aluminum', 2700), ('steel', 7850)]
steel
```

Sorting density values separately from names can produce a neat but false material table. A key function selects the comparison property while the operation moves each whole record. A stable sort keeps the original relative order of records with equal keys. Stability is not the same as numerical correctness or uniqueness; it is a tie-order promise.

12.8. Selection sort exposes repeated minimum searches

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 172.

At position i, selection sort scans the remaining suffix to locate a minimum and then swaps it into position. The sorted prefix grows by one position on each outer pass. The review implementation begins with a copy, so it does not reorder the original outer list.

```python
# Search the full unsorted suffix before fixing the next output position.
def selection_sort(values):
    """Return a sorted copy using selection sort; assume ordered values."""
    result = values.copy()  # Preserve the input sequence.
    for start in range(len(result) - 1):
        smallest = start
        for index in range(start + 1, len(result)):
            if result[index] < result[smallest]:
                smallest = index
        result[start], result[smallest] = result[smallest], result[start]
    return result
```

Code 12.7: Selection sort returning a new outer list.

The key comparison count is $(n-1)+(n-2)+\cdots+1 = n(n-1)/2$. The long-distance swaps in this standard version do not guarantee stability. The copy contributes output storage even if the sorting loop itself uses only a fixed number of scalar indices.

12.9. Merge sort combines already sorted halves

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 173–174.

Merge sort returns a copy immediately for zero or one item, recursively sorts the two halves, and merges them. During a merge, the next smallest remaining head is appended to the result. Once either input is exhausted, its partner's unconsumed suffix is already in order and can be appended.

```python
# Preconditions: both inputs are already sorted and mutually comparable.
def merge(left, right):
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:  # Left item wins an equal-value tie.
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])  # Append items not visited by the joint loop.
    merged.extend(right[j:])
    return merged

def merge_sort(values):
    # This function completes the structure displayed in the review.
    if len(values) <= 1:
        return values.copy()
    mid = len(values) // 2
    left = merge_sort(values[:mid])
    right = merge_sort(values[mid:])
    return merge(left, right)

print(merge([1, 3, 3], [2, 3, 4]))
print(merge_sort([3, 1, 2, 1]))
```

Code 12.8: Complete worked merge-sort implementation from the review's two displayed stages.

```
[1, 2, 3, 3, 3, 4]
[1, 1, 2, 3]
```

Taking the left item on equality preserves the original relative order of equal-key items across the split. Together with stable recursive halves, this produces a stable merge. For balanced splits, logarithmically many levels each handle linear total merging work, giving $\Theta(n \log n)$ under the stated comparison-cost model. Copies, slices, and result lists consume memory; the recursion depth alone is not the whole storage cost.

12.10. Choose for the complete workload

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 175.

A binary search can reduce candidate comparisons, but sorting data for one query may cost more than a single linear scan. For many repeated queries, preprocessing may be worthwhile, provided the order remains valid and original identities are retained. Comparing methods fairly requires equivalent output contracts, the same data assumptions, and explicit preparation and memory costs.

12.11. Practice: attempt before consulting the solutions

12.11.1. M12-Q1: Index zero is a successful answer

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 176.

Predict the results. Why is `if index:` misleading? Which occurrence does this search return?

```python
from review_models import linear_search
index = linear_search(["A", "B", "A"], "A")
print(index, bool(index), index is not None)  # Three different questions.
print(linear_search(["A", "B"], "C"))  # No successful position exists.
```

Code 12.9: Practice M12-Q1. Predict before running.

Worked solution: page 70

12.11.2. M12-Q2: Trace a stable sort by one key

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 178.

Predict both output lines. Why does B remain before C? Did the original list change?

```python
records = [("A", 2), ("B", 1), ("C", 1)]
def key(record):
    return record[1]  # No extra tie-breaker is introduced.

print(sorted(records, key=key))
print(records)  # sorted() did not replace any input position.
```

Code 12.10: Practice M12-Q2. Predict before running.

Worked solution: page 70

12.11.3. M12-Q3: Trace the merge, including ties

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 180.

Merge sorted `left = [1, 3, 3]` and `right = [2, 3, 4]`, taking from the left on ties. Give the output and explain why omitting the leftover tail violates the contract.

Worked solution: page 70

12.12. Worked solutions and reasoning

M12-Q1: worked solution

```
0 False True
None
```

The first `"A"` is at index zero. Zero is false in a Boolean test but is a legitimate successful position; `index is not None` is the appropriate success test for this interface. The source-order linear scan returns the first occurrence. The absent `"C"` gives `None`, after all positions have failed.

Takeaway

An absence policy must be interpreted explicitly, not replaced with a generic truthiness test.

Return to question M12-Q1 (page 69).

M12-Q2: worked solution

```
[('B', 1), ('C', 1), ('A', 2)]
[('A', 2), ('B', 1), ('C', 1)]
```

The key examines only the numeric field. B and C both have key 1, so the stable sort preserves their original relative order. A follows with key 2. `sorted` constructs a new outer list; it does not reorder `records`. Adding the name as a second key would introduce a new tie policy and should be done only when required.

Takeaway

Sort complete records and distinguish stability from an explicitly added tie-breaker.

Return to question M12-Q2 (page 69).

M12-Q3: worked solution

Take left 1, right 2, left 3, and left 3. The left side is exhausted, so append the remaining right tail `[3, 4]`. The final sequence is `[1, 2, 3, 3, 3, 4]`. Omitting the tail would produce an ordered prefix but lose two input elements. Sorted order alone does not establish a correct sort: preservation of items and their multiplicities is also required.

Takeaway

Merge progress and preservation must both be justified.

Return to question M12-Q3 (page 70).

12.13. Search-and-sort checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 181.

State what is returned, why each discarded candidate is impossible, how absence is represented, and what order means. Test empty and singleton inputs, endpoints, duplicates, and missing targets. For sorting, also test multiplicity, stable ties when promised, record integrity, and preservation of the original data.

13 Programming Beyond Python

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 182–196.

After reviewing this module

Transfer the engineering model and its tests rather than punctuation alone; recognize important numeric, indexing, array, and reference differences; and separate implementation agreement from model validation.

13.1. Keep the problem and evidence fixed

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 183.

A translation changes the expression of a computation, not the obligation to solve the stated problem. Inputs, units, assumptions, boundaries, expected outputs, failure behavior, and non-mutation promises should be written before choosing target-language syntax.

Concepts such as decomposition, iteration, testing, searching, and data integrity transfer broadly. Exact operators, indexing origins, collection shapes, equality rules, and calling conventions may not. A program can compile successfully and still change the mathematics because one operator has a different meaning.

The review interprets the course-outline entry "C+" as C++. That interpretation is retained explicitly. The module is a reading and transfer exercise, not a requirement to master several new languages or install every tool.

13.2. Distinguish the layers of a programming environment

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 184.

A language defines program expressions and their meanings. A library supplies reusable operations. An editor is where code can be written and inspected. A runtime or interpreter executes supported representations, while a compiler translates code according to its toolchain. An IDE can integrate several of these services without becoming the language itself.

Performance depends on the algorithm, implementation, libraries, input, and environment. A language label alone does not prove a particular program is faster. Likewise, using a compiled language does not automatically enforce engineering units or prevent an incorrect model assumption.

13.3. Integer division can change the answer before assignment

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 185.

In Python, `7 / 2` gives 3.5 and `7 // 2` gives 3. In the displayed C++ and Java examples, `7 / 2` is integer division because both operands are integers. Assigning that already computed result to a floating-point variable yields 3.0, not 3.5. A floating-point operand changes the division itself.

```python
# The first expression converts an already computed floor quotient.
a = float(7 // 2)
b = 7 / 2  # True division uses the fraction for ordinary Python numbers.
print(a, b)
```

Code 13.1: Read the operand types before interpreting division.

```
3.0 3.5
```

```cpp
// Worked wrapper for the division comparison in the review.
#include <iostream>
int main() {
    double a = 7 / 2;    // Integer operands are divided before assignment.
    double b = 7.0 / 2;  // A floating-point operand preserves the fraction.
    std::cout << std::fixed;
    std::cout.precision(1);
    std::cout << a << " " << b << "\n";
    return 0;  // Successful program termination.
}
```

Code 13.2: C++ demonstration: conversion after division cannot recover the discarded fraction.

```java
// Worked wrapper for the division comparison in the review.
public class Division {
    public static void main(String[] args) {
        double a = 7 / 2;    // Integer division occurs first.
        double b = 7.0 / 2;  // Floating-point division occurs here.
        System.out.println(a + " " + b);
    }
}
```

Code 13.3: Java demonstration of the same expression-level distinction.

The division checks are supplied as small complete programs, so compilation and execution can be distinguished from reading an isolated fragment. Their fixed outputs are checked in the companion verification resources. No general language-speed conclusion follows from them.

13.4. Indexing, shape, and array operations

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 186.

Python sequence indices begin at zero, while MATLAB array indices begin at one. A Python `range(1, 6)` excludes its upper stop and yields 1 through 5; the MATLAB integer colon expression `1:5` includes both shown endpoints. MATLAB's `end` can identify a final index in indexing expressions.

MATLAB distinguishes elementwise powers and products from matrix operations. `A.^2` squares elements, while `A^2` is a matrix power. MATLAB `A.*B` is elementwise multiplication, whereas `A*B` is matrix multiplication when dimensions permit. For NumPy arrays, the corresponding familiar distinctions are `A * B` and `A @ B`. Plain Python lists are not NumPy arrays; their operators must not be read as elementwise numeric operations.

```m
% Reading exercise: MATLAB was not executed during preparation.
a = [1 2 3];              % Numeric row vector: shape matters.
squared = a.^2;           % Elementwise power, not matrix power.
assert(isequal(squared, [1 4 9])); % Expected, not captured, result.
first = a(1);             % MATLAB uses one-based indexing.
last = a(end);
disp(squared);
```

Code 13.4: MATLAB reading exercise with derived expected values and native checks.

Execution status

The MATLAB programs are supplied for reading and native execution by students. MATLAB execution is not claimed for the prepared package. Expected MATLAB values are derived from the displayed operations, not presented as captured MATLAB output.

13.5. Read blocks and interfaces from their purpose outward

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 187.

Python uses indentation and colons to define many blocks. MATLAB closes corresponding structures with `end`; C++ and Java commonly use braces. When reading a translated function, identify the inputs, current state, condition, update, returned result, and failure route before treating additional syntax as a new algorithm.

Declared numeric types do not attach physical dimensions. A value in pascals can be passed where kilopascals were intended in a statically typed program just as it can in Python. Documented interfaces and independent unit tests remain necessary.

13.6. Equality, aliases, and copies need language-specific attention

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 188.

Python string `==` compares content, whereas `is` asks about identity. Java string content should be compared with the intended content-comparison method rather than relying on reference equality. C++ `std::string` equality compares string content.

Assignment of a Python list reference creates an alias; assignment of a Java array reference also refers to the same array rather than copying its elements. A standard C++ vector copied by value has its own element storage. That last statement does not imply deep copying of every possible object referred to by vector elements. Preserve the source exercise's simple value-storage assumptions when making comparisons.

13.7. Translate a cooling recurrence, not just its characters

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 189.

The reviewed update is

$$T_{i+1} = T_i - \Delta t\, k(T_i - T_{\text{ambient}}).$$

Both implementations must begin with the same initial temperature, use compatible units for k and Δt, perform the same number of updates, and preserve the same set of samples. Storing the initial value plus 100 updated values produces 101 samples, not 100.

Worked extension of the review

The source review gives the symbolic recurrence and sample-count comparison. The following runnable setup supplies explicitly illustrative values: $T_0 = 100°\text{C}$, $T_{\text{ambient}} = 20°\text{C}$, $k = 0.1$ s^{-1}, $\Delta t = 0.1$ s, and 100 updates. These are a worked completion, not additional measured data or an assessment requirement.

```python
# Additional illustrative setup for the review's symbolic recurrence.
# This is a fixed-rate, well-mixed teaching model, not measured data.
temperature_c = 100.0
ambient_c = 20.0
rate_per_s = 0.1
step_s = 0.1
updates = 100
times_s = [0.0]
temperatures_c = [temperature_c]  # Retain the initial sample.
for index in range(updates):
    temperature_c -= step_s * rate_per_s * (temperature_c - ambient_c)
    times_s.append((index + 1) * step_s)
    temperatures_c.append(temperature_c)
print(len(temperatures_c))  # 100 updates plus the initial sample.
print(f"Final stored model value = {temperatures_c[-1]:.6f} deg C")
```

Code 13.5: Python implementation of the labelled illustrative cooling setup.

```
101
Final stored model value = 49.282587 deg C
```

```m
% Worked completion of the symbolic cooling recurrence in the review.
% These added parameters match m13_cooling.py; no MATLAB run is claimed.
ambient_c = 20.0;
rate_per_s = 0.1;
step_s = 0.1;
updates = 100;
times_s = (0:updates) * step_s; % 101 stored locations.
temperatures_c = zeros(1, updates + 1); % Preallocate a row vector.
temperatures_c(1) = 100.0;      % First slot stores the initial state.
for index = 1:updates
    temperatures_c(index + 1) = temperatures_c(index) ...
        - step_s * rate_per_s * (temperatures_c(index) - ambient_c);
end
assert(numel(temperatures_c) == 101); % Check the sample-count contract.
disp(temperatures_c(end));
```

Code 13.6: MATLAB version of the same illustrative update and storage contract.

Three checks remain separate. First, did each implementation calculate the stated recurrence? Second, how closely does that discrete recurrence approximate the continuous model? Third, is the assumed cooling model suitable for the physical situation? Agreement between Python and MATLAB would support the first comparison, not automatically the second or third.

13.8. Use assistance without transferring responsibility

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 190.

An AI-assisted translation should start with the source contract and test cases. Inspect changed operators, conversions, index boundaries, returned types, and failure behavior. Run the original known-answer and rejection cases in the target language, then explain differences rather than accepting plausible output alone.

Follow the permitted-assistance policy for the activity and remove confidential or personal data before sharing code. The review supplies no new AI policy or examination permission. Its emphasis is that generated code remains subject to the same verification and interpretation obligations.

13.9. Practice: attempt before consulting the solutions

13.9.1. M13-Q1: Which value survives assignment?

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 191.

In C++ or Java, compare `double a = 7 / 2;` with `double b = 7.0 / 2;`. Give corresponding Python expressions yielding floating-point 3.0 and 3.5.

Worked solution: page 77

13.9.2. M13-Q2: An array operation is not a typographic detail

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 193.

The task squares each element of a MATLAB numeric row vector. Why use `a.^2` rather than `a^2`? What must be checked when translating from Python?

Worked solution: page 77

13.9.3. M13-Q3: Build a translation test set

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 195.

A translated stress function takes force in N and positive area in mm^2, returns MPa, and rejects nonpositive area. Give tests that check meaning rather than source-code resemblance.

Worked solution: page 77

13.10. Worked solutions and reasoning

M13-Q1: worked solution

The integer operands in the first native expression are divided before assignment, producing the integer quotient 3. Converting that quotient for storage in a `double` gives 3.0, not 3.5. The floating-point operand in the second expression gives 3.5 directly. Python expressions are `float(7 // 2)` and `7 / 2`. Converting an already truncated quotient cannot reconstruct its missing fraction.

Takeaway

Trace expression evaluation before interpreting the destination type.

Return to question M13-Q1 (page 76).

M13-Q2: worked solution

The dot selects elementwise exponentiation. Matrix power is a different operation and generally requires a square matrix; a nonsquare row vector cannot be squared as a matrix. Check whether the Python object is a NumPy array or a plain list, because a list does not support NumPy's elementwise power operation. Verify shape and the intended mathematical operation as well as the numeric values.

Takeaway

A similar symbol in another language may represent a different mathematical operation.

Return to question M13-Q2 (page 76).

M13-Q3: worked solution

Use $(10000, 500) \to 20$ MPa, $(0, 500) \to 0$ MPa, and $(20000, 500) \to 40$ MPa. Check that both zero and negative area are rejected. These tests cover an independent absolute answer, a zero result, scaling, and invalid domain boundaries. Add wrong types or nonfinite proposals only when both language implementations promise those checks. Meaningful failure behavior need not use identical exception text across languages.

Takeaway

Translation tests should compare the shared contract, including units and failures.

Return to question M13-Q3 (page 77).

13.11. Language-transfer checkpoint

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 196.

Write down what must stay unchanged, then identify language details that could violate that promise. Translate one small component, test it, and only then reconnect the larger workflow. Model understanding, data meaning, and independent evidence remain the transferable skills.

14 Integrated Engineering Review

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 197–213.

After reviewing this module

Follow one dataset from raw text to an explained result, preserve original identities and rejected evidence, distinguish data validity from interval compliance, and test the complete workflow as well as its components.

14.1. State the complete report contract

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 198.

The integrated example processes temperature records with exactly three comma-separated fields: a nonempty identifier, numerical text, and the exact supported unit code `"C"`. Accepted values must be finite. The stated normal interval is $20 \leq T \leq 30$ in degrees Celsius, including both endpoints.

Each accepted record retains its original input index, identifier, value, and category. Each rejected record retains its original index, raw text, and rejection explanation. Rejected data are not replaced by zero, and accepted readings outside the normal interval are retained rather than hidden.

The parser does not establish calibration quality, global physical plausibility, identifier uniqueness, or a required `"T-"` prefix. These would be additional rules. Do not silently attribute them to the source implementation.

14.2. Preserve the seven raw records

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 199.

Index	Raw record	Expected interpretation
0	`T-01,19.5,C`	Finite Celsius value, below the normal interval.
1	`T-02,20.0,C`	Included lower boundary.
2	`T-03,25.0,C`	Interior normal value.
3	`T-04,30.0,C`	Included upper boundary.
4	`T-05,30.5,C`	Finite Celsius value, above the normal interval.
5	`T-06,bad,C`	Nonnumeric value text: reject.
6	`T-07,300.0,K`	Unsupported unit code: reject rather than silently convert.

The values are a synthetic teaching dataset. The last record is rejected by the parser's unit contract, not by a claim that 300 K is physically invalid. Supporting Kelvin conversion would require an explicit extension of the interface.

14.3. Decompose the pipeline into visible responsibilities

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 200.

Parsing extracts a record's fields. Validation checks whether they meet the input contract. Classification applies the stated interval. Summarization calculates the accepted-data mean. Ranking reorders complete accepted records for one reporting purpose. Testing checks both individual components and the relationships between them. This decomposition is not permission

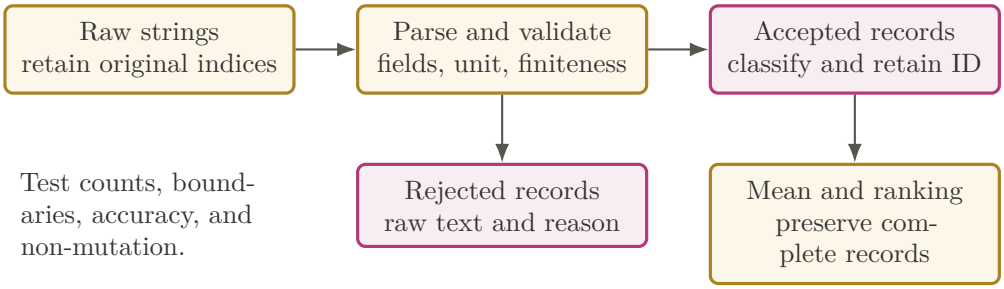


Figure 14.1.: The integrated review workflow. A rejected record remains evidence with an index and a reason; it is not a fabricated numerical observation.

to lose context between stages. Every numerical value must retain its identity and unit, and every rejection must remain distinguishable from an ordinary reading.

14.4. Parse deliberately and reject unsupported inputs

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 201–202.

The parser first splits on commas and checks the field count. It trims surrounding whitespace from the identifier and unit, checks that the identifier is not empty, and checks the exact supported unit. Numerical text is then converted and checked for finiteness.

```python
# Parsing validates a record; operating limits are applied later.
def parse_temperature(text):
    """Parse exactly 'identifier,value,C'; no automatic unit conversion."""
    parts = text.split(",")
    if len(parts) != 3:
        raise ValueError("expected three comma-separated fields")
    name, value_text, unit = parts
    name, unit = name.strip(), unit.strip()
    if not name or unit != "C":
        raise ValueError("need a nonempty ID and the exact unit C")
    value = float(value_text)  # Malformed numbers raise ValueError.
    if not math.isfinite(value):
        raise ValueError("temperature must be finite")
    return name, value
```

Code 14.1: The complete review parser and its explicitly limited data contract.

```python
# Numeric type, representable range, and finiteness are separate tests.
def finite_number(value, name="value"):
    """Convert a built-in int/float to a finite float, excluding bool."""
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError(f"{name} must be an int or float, not bool")
    try:
        result = float(value)  # Make the supported numeric range explicit.
    except OverflowError as error:
        raise ValueError(f"{name} is too large for this model") from error
    if not math.isfinite(result):
        raise ValueError(f"{name} must be finite")
    return result
```

Code 14.2: Shared numeric validation used by the review's checked functions and classes.

Converting text to a floating-point value can succeed for special representations such as infinity. This is why conversion and finite-value validation are separate operations. The helper rejects Boolean input when called directly, even though a Boolean has a numeric relationship to integers in Python.

14.5. Keep accepted and rejected paths separate

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 203–204.

The driver checks the supplied limits, then visits each raw record with `enumerate`. A parsing `ValueError` adds a rejection record. A successfully parsed reading is classified and added to the accepted records. The driver assumes a list of record strings; it does not claim to accept arbitrary objects as record text.

```python
# The caller, not this simple classifier, validates the inputs.
def classify_temperature(temp_c, low_c=20.0, high_c=30.0):
    """Classify finite values; caller supplies ordered limits in deg C."""
    if temp_c < low_c:
        return "too low"
    elif temp_c <= high_c:
        return "normal"  # Both endpoints belong to the normal interval.
    return "too high"
```

Code 14.3: The same inclusive interval classifier used elsewhere in the review.

```python
# Preserve original positions even when accepted records form a new list.
def classify_records(raw_records, lower=20.0, upper=30.0):
    """Return accepted, rejected records and mean; preserve source order.

    Records are (original_index, identifier, value, classification).
    Rejections are (original_index, raw_text, error_message).
    An empty accepted set gives mean=None, never a fabricated zero.
    """
    lower, upper = finite_number(lower), finite_number(upper)
    if lower > upper:
        raise ValueError("limits must be ordered")
    accepted, rejected = [], []
    # BEGIN pipeline_loop
    # Keep rejected records visible; never invent replacement readings.
    for index, raw in enumerate(raw_records):
        try:
            name, value = parse_temperature(raw)
        except ValueError as error:
            rejected.append((index, raw, str(error)))
        else:
            category = classify_temperature(value, lower, upper)
            accepted.append((index, name, value, category))
    # END pipeline_loop
    mean = None
    if accepted:
        mean = sum(record[2] for record in accepted) / len(accepted)
    if mean is not None and not math.isfinite(mean):
        raise ValueError("mean overflow; review the numeric scale")
    return accepted, rejected, mean
```

Code 14.4: The complete reviewed driver, including the no-accepted-readings result.

The valid data produce categories too low, normal, normal, normal, and too high at original indices 0 through 4. Rejected indices 5 and 6 remain in a separate list. The accounting identity is

$$N_{\text{accepted}} + N_{\text{rejected}} = N_{\text{raw}} = 7.$$

It checks that every supplied record followed exactly one route. It does not by itself prove each route's decision was correct; test the known individual cases as well.

14.6. Interpret the mean alongside the individual observations

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 205–206.

The accepted values sum to

$$19.5 + 20.0 + 25.0 + 30.0 + 30.5 = 125.0^\circ\text{C}, \qquad \bar{T} = 125.0/5 = 25.0^\circ\text{C}.$$

The mean falls inside the normal interval, but readings at indices 0 and 4 do not. Reporting only "average acceptable" would omit the two individual warnings. If no records are accepted, the mean is `None`; a numeric mean of zero would invent information. A graph needs the same provenance rules as a table. Label axes and units, preserve original positions, show the criterion, and explain the result. A visualization is not an excuse to omit rejected records from the narrative.

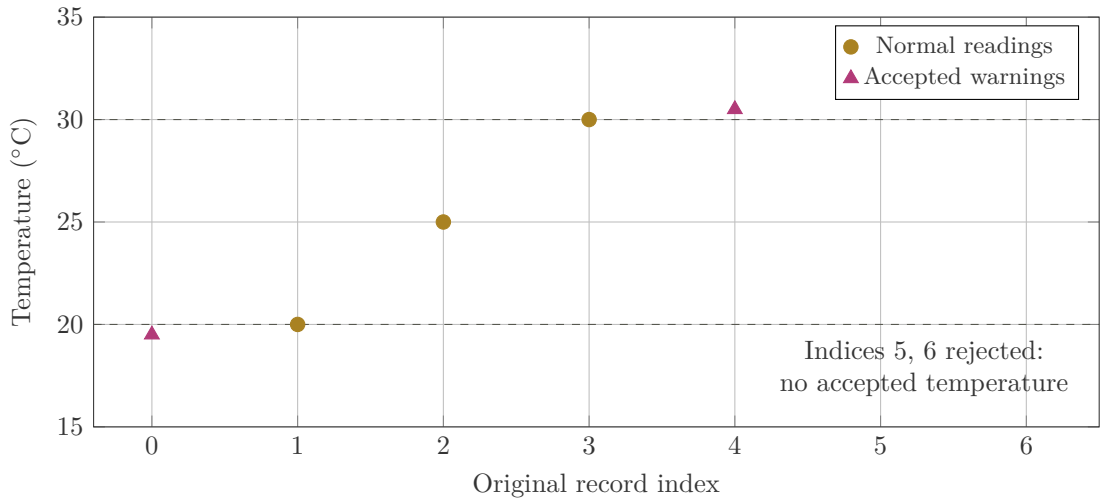


Figure 14.2.: Recreated graphical evidence for the seven-record review dataset. Accepted readings use their original input indices. Rejected records 5 and 6 have no accepted temperature to plot; they are identified by an annotation rather than by a fictitious zero-temperature point.

14.7. Rank complete accepted records

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 207.

A numerical key can sort the accepted tuples by their temperature while moving the entire tuple. Descending order gives identifiers T-05, T-04, T-03, T-02, and T-01. The original indices are still available, so ranking does not falsely imply a new acquisition order.

```
# Return only the comparison key; the entire record is retained.
def temperature_key(record):
    """Select temperature without separating it from its identifier."""
    return record[2]
```

Code 14.5: Select the comparison property while keeping records intact.

```
from review_models import RAW_RECORDS, classify_records, temperature_key
# Processing retains original indices and reports failed records separately.
accepted, rejected, mean = classify_records(RAW_RECORDS)
ranked = sorted(accepted, key=temperature_key, reverse=True)
print([record[1] for record in ranked])  # IDs follow their full records.
print([record[0] for record in rejected])
```

Code 14.6: Rank complete records and identify rejected original indices.

```
['T-05', 'T-04', 'T-03', 'T-02', 'T-01']
[5, 6]
```

The two printed lists identify the ranked accepted IDs and the rejected original indices. The mean, counts, warning records, and rejection reasons are separate results discussed above. None should be silently substituted for the others.

14.8. Test the complete route through the program

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 208.

The basic verification checks five acceptances, two rejections, three normal readings, warnings at original indices 0 and 4, a mean of 25.0, and no change to the raw input. Component tests separately check parsing and boundary behavior.

```
from review_models import RAW_RECORDS, classify_records
before = RAW_RECORDS.copy()  # Save the original input for comparison.
accepted, rejected, mean = classify_records(RAW_RECORDS)
assert len(accepted) == 5 and len(rejected) == 2
assert len(accepted) + len(rejected) == len(RAW_RECORDS)
assert mean == 25.0  # Independent sum of the five accepted values.
assert [r[0] for r in accepted if r[3] != "normal"] == [0, 4]
assert RAW_RECORDS == before  # The pipeline must not overwrite input.
print("Integrated checks passed")  # Added test-driver confirmation.
```

Code 14.7: Independent checks of counts, mean, warning locations, ranking, and raw-data preservation.

```
Integrated checks passed
```

Additional cases follow from the same contract: an empty input, all rejected records, repeated identifiers, exact interval boundaries, malformed field counts, and nonfinite numeric text. Repeated IDs are not automatically rejected by this parser; a uniqueness test would need a separately stated requirement.

14.9. Explain efficiency without losing string-processing cost

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 209.

For n bounded-length records, parsing and classification require a linear pass. If k records are accepted, their mean needs linear work in k, and comparison sorting uses an $O(k \log k)$ worst-case growth description under bounded-cost comparisons. Retaining accepted and rejected reports also requires storage that grows with their sizes.

If record lengths vary substantially, include total text length in the model rather than calling each parse a fixed-cost operation. The practical pipeline includes output formatting, copying, and error recording as well as numerical arithmetic. An optimization must preserve all required reports, not only the mean.

14.10. Practice: attempt before consulting the solutions

14.10.1. INT-Q1: Account for every record

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 210.

For the seven-record dataset, report accepted, rejected, normal, and warning counts and the mean. Why are warning and rejection different even though both counts equal two?

Worked solution: page 85

14.10.2. INT-Q2: Choose a cross-language regression set

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 212.

The complete report is translated to another language. Which outputs and side effects must be compared before claiming that behavior is preserved?

Worked solution: page 85

14.11. Worked solutions and reasoning

INT-Q1: worked solution

Five records are accepted and two are rejected. Of the five accepted readings, three are normal and two are warnings. Their sum is 125.0°C, giving mean 25.0°C. The warnings are finite Celsius readings outside the stated interval. The rejected records fail conversion or the unit contract. The equality of the two counts is incidental: warning is a subset of accepted data, not another word for rejection.

Takeaway

Use raw = accepted + rejected and accepted = normal + warnings as separate accounting identities.

Return to question INT-Q1 (page 84).

INT-Q2: worked solution

Compare accepted record contents and original indices, classification at exact limits, rejected indices and meaningful failure reasons, the mean within a stated tolerance, ranking and tie policy, and preservation of the raw list. Include empty and all-rejected cases. Check repeated identifiers without inventing a uniqueness restriction. Similar-looking code or agreement only on the mean leaves much of the contract untested.

Takeaway

An integration test must cover the information retained and lost along every route.

Return to question INT-Q2 (page 85).

15 Mixed-Topic Practice

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 214–234.

After reviewing this module

Combine concepts without relying on a chapter heading to suggest the answer. Explain control flow, data meaning, references, algorithmic work, and failure behavior in one coherent argument.

The ten mixed questions deliberately cross topic boundaries. For example, an assignment can involve a method's returned value and a shared object at the same time; a compact method call can hide a full scan; and an insertion position can be valid without demonstrating a successful search.

Make an initial attempt before consulting the worked answer. For a trace, write intermediate states. For a complexity claim, define the input size and count the relevant work. For a numerical claim, state the model and unit. For a failure case, describe both the exception and the state left behind. A correct option or final number without that explanation is not strong evidence of transferable understanding.

The questions retain their source identifiers X01–X10. They are formative practice, not announced examination questions, grading criteria, or a promise about an examination format.

15.1. Practice: attempt before consulting the solutions

15.1.1. X01: Choose the strongest correctness claim

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 215.

Choose and justify one statement. A: the program runs, so its engineering model is valid. B: selected passing tests prove every input correct. C: the result satisfies selected independent checks under stated assumptions. D: more decimal places prove greater physical accuracy.

Worked solution: page 88

15.1.2. X02: A default parameter and a local name

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 217.

Write both output lines. Which `gain` is global and which is local? Does a keyword argument permanently change the default?

```
gain = 10  # This name is separate from the function parameter.
def scale(value, gain=2):
    return value * gain  # Use the argument for this call.

print(scale(3), scale(3, gain=4))
print(gain)  # The outer value was not reassigned.
```

Code 15.1: Practice X02. Predict before running.

Worked solution: page 88

15.1.3. X03: A tuple can contain a mutable object

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 219.

Predict the tuple, identify the changed object, and explain whether `record[0] = "B"` is permitted.

```
record = ("A", [1, 2])
record[1].append(3)  # Mutate the contained list, not a tuple slot.
print(record)
```

Code 15.2: Practice X03. Predict before running.

Worked solution: page 89

15.1.4. X04: Select an input-space guarantee

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 221.

Choose and justify one observation. A: the iteration counter reached its maximum. B: the displayed residual has six decimal places. C: a continuous root remains in a bracket of width 0.002 and the estimate is its midpoint. D: consecutive guesses rounded to the same displayed value.

Worked solution: page 89

15.1.5. X05: Repair invalid data without falsifying it

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 223.

A program catches every conversion exception and substitutes 0.0. Its average lies inside the normal interval. Explain the defect and propose a report retaining the failed record.

Worked solution: page 89

15.1.6. X06: Return value versus shared state

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 225.

Predict the three values. Why is the update visible through both names, and why is `answer None`?

```
from intro_sensor import IntroSensor
a = IntroSensor("A", "deg C", 25.0, 20.0, 30.0)
b = a  # Alias: both names select the same sensor.
answer = b.update(35.0)  # Mutates that sensor; returns None.
print(a.value, b.value, answer)
```

Code 15.3: Practice X06. Predict before running.

Worked solution: page 89

15.1.7. X07: Classify a hidden nested scan

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 227.

A list of n records is traversed once. Each pass calls a method that recomputes a sum of all n readings. Choose the tight class $\Theta(1)$, $\Theta(\log n)$, $\Theta(n)$, or $\Theta(n^2)$ under bounded-cost arithmetic.

Worked solution: page 89

15.1.8. X08: An insertion position needs a bounds check

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 229.

Predict both outputs. Why is `values[pos]` not evaluated? What would direct access without the guard do?

```
from bisect import bisect_left
values = [10, 20, 30]
pos = bisect_left(values, 40)  # A valid insertion slot can be len(values).
found = pos < len(values) and values[pos] == 40
print(pos)
print(found)  # The out-of-range access was never evaluated.
```

Code 15.4: Practice X08. Predict before running.

Worked solution: page 89

15.1.9. X09: One lookup versus many lookups

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 231.

For one target in an unsorted list, a classmate proposes sorting before binary search because binary search is logarithmic. What is missing? When might preprocessing be worthwhile?

Worked solution: page 90

15.1.10. X10: Interpret a failed update correctly

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 233.

Choose and justify one statement. A: every out-of-limit reading must be discarded. B: a rejected invalid proposal leaves previous valid state unchanged when the setter promises that contract. C: a setter returning `None` performed no update. D: a leading underscore prevents all external access.

Worked solution: page 90

15.2. Worked solutions and reasoning

X01: worked solution

C is the defensible claim. Execution alone does not validate the physical model. A finite test suite can miss cases or contain incorrect expectations. Formatting more digits changes the display, not the quality of the measurements or assumptions. State both what has been checked and the limitations that remain.

Return to question X01 (page 86).

X02: worked solution

```
6 12
10
```

The first call uses the default local argument 2 and returns 6; the second uses the explicitly supplied 4 and returns 12. The separate outer `gain` remains 10. A supplied value replaces the default choice for that particular call, not the function's default definition.

Return to question X02 (page 86).

X03: worked solution

```
('A', [1, 2, 3])
```

The contained list changes to `[1, 2, 3]`, so the tuple displays `('A', [1, 2, 3])`. Neither tuple slot has been replaced. Assigning to `record[0]` would attempt slot replacement and raises `TypeError`. Immutability of a container's references is not recursive immutability of all objects it refers to.

Return to question X03 (page 87).

X04: worked solution

C yields an exact-arithmetic midpoint error bound of 0.001 in the input variable's unit. Reaching a budget is a stopping event, not an accuracy guarantee. Display precision and repeated rounded values do not establish a small error. The bracket guarantee must still be interpreted with its continuity, invariant, and arithmetic assumptions.

Return to question X04 (page 87).

X05: worked solution

The replacement invents a measured value and can distort the mean and warning categories. Catch the expected conversion failure specifically and retain the original record index, raw text, and reason. Exclude it under an explicit policy and report accepted and rejected counts with the summary. A normal-looking average does not retrospectively make the input handling correct.

Return to question X05 (page 87).

X06: worked solution

```
35.0 35.0 None
```

The assignment `b = a` gives a second name to the same sensor. Updating through `b` therefore changes the reading observed through both references to 35.0. The method does not explicitly return a value, so `answer` is `None`. Mutation of an object and a returned result are independent properties of a call.

Return to question X06 (page 87).

X07: worked solution

The total work is $\Theta(n^2)$: there are n outer passes, and each performs an n-item sum. The method's short interface hides the inner scan. A cached or hoisted subtotal is valid only when its underlying data and the required result do not change during the outer loop.

Return to question X07 (page 87).

X08: worked solution

```
3
False
```

The insertion position is 3, equal to the list length. The first part of `and` is false, so short-circuit evaluation skips `values[pos] == 40`. The final result is `False`. A direct read of `values[3]` raises `IndexError`: an insertion slot beyond the final item is not an existing item.

Return to question X08 (page 88).

X09: worked solution

The argument omits sorting cost, memory, and preservation of original positions. A single linear scan costs $O(n)$ in the worst case, whereas general sorting first normally introduces $O(n \log n)$ work before lookup. Repeated queries on largely unchanged data can amortize preparation. Compare complete workloads, including later updates and the required first-match, arbitrary-match, or insertion-position policy.

Return to question X09 (page 88).

X10: worked solution

B states the failure contract. In this monitoring model, valid out-of-limit readings are retained and flagged rather than erased. A setter can change state and return `None`. A leading underscore communicates a non-public convention but is not enforced security. Judge each claim against the documented quantity and interface.

Return to question X10 (page 88).

16 Final Checks and Targeted Revision

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 235–240.

16.1. Judge readiness by what you can explain

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 236.

The review is successful when you can use the ideas independently, not merely recognize a familiar slide. Choose an example you have not recently rehearsed and demonstrate the following abilities.

1. Trace a program with a new input and explain every change in state.
2. Write or repair a focused function with a clear contract.
3. Select tests that expose boundaries, missing data, and unintended mutation.
4. Explain why a loop or recursion terminates and whether it succeeded.
5. Justify algorithm cost and preconditions, and preserve meaning across languages.

When an example spans several modules, identify the interface at every connection. A correct unit conversion is useful only if the receiving function gets the converted quantity. A correctly sorted list is useful only if it retains the intended record associations.

16.2. Use an error log to direct the next attempt

Lecture-slide alignment: Revised Week 14 self-study PDF, p. 237.

Do not spend equal time rereading every topic. Record the misconception that caused a wrong answer, make the smallest conceptual repair, and then try a changed example without its solution.

Record	Example from the source review
Exercise ID	M05-Q2: aliasing.
My prediction	I expected the raw list to remain unchanged.
Observed result	The raw list changed through its alias.
Underlying misconception	I treated assignment as a copy.
Repair	Draw names pointing to objects; distinguish rebinding from mutation.
Retest	Try a different flat list, then a nested list with an outer copy.

A new input helps distinguish understanding of the rule from memory of an answer. The contents, bookmarks, question identifiers, and page index allow targeted return to the relevant module. No three-lecture pacing schedule is imposed on this review.

16.3. Source basis and explicit distinctions

Lecture-slide alignment: Revised Week 14 self-study PDF, pp. 238–240.

The revised 240-page Week 14 self-study PDF is the controlling source for these notes. Its topic sequence follows the instructor's fourteen-week outline, and its examples consolidate the preceding course materials. The earlier 87-page Week 14 three-lecture version does not control this reading document.

The source itself openly corrects `"PUMP-107A"[::2]` to `"PM-0A"`, retains $F(0) = F(1) = 1$, supplies the complete nine-call $F(4)$ tree, and distinguishes the introductory `IntroSensor` from the checked review `Sensor`. These distinctions are preserved here rather than reconciled silently.

The two recovered companion modules, `review_models.py` and `intro_sensor.py`, are included unchanged. Their printed excerpts and worked drivers receive additional comments where useful. Driver inputs added to make a symbolic pattern executable are labelled as worked completions. The illustrative cooling setup is likewise identified as an addition to the symbolic review, not historical measurement data.

C++ and Java division examples are small native verification exercises. MATLAB code is provided with native checks and derived expected values, but MATLAB execution is not claimed. No timing number is treated as a universal speed result; the timing demonstration intentionally has no fixed reference duration.

16.4. A compact final reference

When you see...	Ask...
An assignment	Which name or attribute is rebound? Which object, if any, is mutated?
A condition	What happens exactly at the boundary? Are units compatible?
A loop	What changes? What ends it? Does the final required item contribute?
A function call	Which arguments bind? What returns? What else changes?
Recursion	Which base case is reached? What shrinks? How are results combined?
A collection	What does order mean? Are fields associated? What is absent?
An object method	Who is the receiver? What state remains after success or failure?
A search	Which candidates remain? Why can others be discarded?
A sort	What is the key? Are ties, duplicates, and complete records preserved?
An efficiency claim	What is input size? Which case and cost model? What is excluded?
A translation	Which contracts and independent tests must stay unchanged?
A numerical result	Which model, evidence, tolerance, and limitations support it?

The course in one sentence

A trustworthy result needs a clear question, meaningful data, a justified method, tested code, and an honest interpretation.

A Source Alignment and Question Index

A.1. Controlling source and use of earlier materials

These notes expand *Programming for Engineers: Comprehensive Course Review, Self-Study and Practice*, the revised 240-page Week 14 PDF previously supplied in this course project. Page references throughout this document refer to its PDF page numbers, not to the older 87-page three-lecture deck.

The original self-study companion contains the same 48 practice questions without solutions. The questions-only workbook supplied with these reading notes retains those identifiers and separates attempts from answers. It is not a new assessment.

The source review credits the instructor's course sequence, the preceding Week 1–13 decks, and Y. Wang's *Programming for Engineers with Python*, version 20260815. This document uses the revised review as its immediate basis; it does not replace that sequence with the manuscript's alternative chapter map or reproduce unrelated material from the wider book.

A machine-readable manifest records the source PDF's SHA-256 digest and the two recovered Python companions. The exact source companions are retained in the source package's `reference` directory. Added comments, drivers, explanatory derivations, tests, and the illustrative cooling setup are identified in the text or example index. No external examination dates, rules, or grading thresholds have been added.

A.2. Topic map

Module	Topic	Source pages
01	Computation, variables, and types	7–18
02	Decisions and repetition	19–32
03	Strings and numerical search	33–50
04	Functions and program design	51–63
05	Lists, tuples, and data integrity	64–78
06	Recursion and dictionaries	79–93
07	Testing, debugging, and failure	94–108
08	Classes, instances, and methods	109–119
09	Interfaces, shared state, and inheritance	120–133
10	Efficiency: counts and growth	134–147
11	Hidden work, recursion, and memory	148–162
12	Searching and sorting	163–181
13	Programming beyond Python	182–196
14	Integrated engineering review	197–213
15	Mixed-topic practice	214–234
16	Final checks and reference	235–240

A.3. Question and worked-answer index

The following links point to pages in *these reading notes*. The source pages permit comparison with the original self-study PDF.

ID	Question page	Solution page	Source Q / A
M01-Q1	4	5	14 / 15
M01-Q2	4	5	16 / 17
M02-Q1	9	10	26 / 27
M02-Q2	10	10	28 / 29
M02-Q3	10	11	30 / 31
M03-Q1	16	17	45 / 46
M03-Q2	16	17	47 / 48
M03-Q3	17	17	49 / 50
M04-Q1	21	22	58 / 59
M04-Q2	21	22	60 / 61
M04-Q3	22	23	62 / 63
M05-Q1	27	28	73 / 74
M05-Q2	28	29	75 / 76
M05-Q3	28	29	77 / 78
M06-Q1	33	34	88 / 89
M06-Q2	34	35	90 / 91
M06-Q3	34	35	92 / 93
M07-Q1	39	40	103 / 104
M07-Q2	39	40	105 / 106
M07-Q3	40	40	107 / 108
M08-Q1	44	45	115 / 116
M08-Q2	45	45	117 / 118
M09-Q1	50	51	128 / 129
M09-Q2	50	51	130 / 131
M09-Q3	50	51	132 / 133
M10-Q1	55	56	144 / 145
M10-Q2	55	56	146 / 147
M11-Q1	62	62	157 / 158
M11-Q2	62	63	159 / 160
M11-Q3	62	63	161 / 162
M12-Q1	69	70	176 / 177
M12-Q2	69	70	178 / 179
M12-Q3	70	70	180 / 181
M13-Q1	76	77	191 / 192
M13-Q2	76	77	193 / 194
M13-Q3	77	77	195 / 196
INT-Q1	84	85	210 / 211
INT-Q2	85	85	212 / 213
X01	86	88	215 / 216
X02	86	88	217 / 218
X03	87	89	219 / 220
X04	87	89	221 / 222
X05	87	89	223 / 224
X06	87	89	225 / 226
X07	87	89	227 / 228

ID	Question page	Solution page	Source Q / A
X08	88	89	229 / 230
X09	88	90	231 / 232
X10	88	90	233 / 234

A.4. Complete slide-page coverage

Every source page is included once in this coverage map. Consecutive pages sharing the same note destination are grouped. A chapter destination covers a divider or overview; a section or practice destination identifies the corresponding detailed treatment.

Source PDF pages	Reading-note destination	Notes page
1–6	Orientation and study guide	i
7	Computation, variables, and types	1
8	Reading explanation	1
9	Reading explanation	1
10	Reading explanation	2
11	Reading explanation	3
12	Reading explanation	3
13	Reading explanation	1
14	Practice M01-Q1	4
15	Worked answer M01-Q1	5
16	Practice M01-Q2	4
17	Worked answer M01-Q2	5
18	Reading explanation	5
19	Decisions and repetition	6
20	Reading explanation	6
21	Reading explanation	7
22	Reading explanation	6
23	Reading explanation	7
24	Reading explanation	8
25	Reading explanation	8
26	Practice M02-Q1	9
27	Worked answer M02-Q1	10
28	Practice M02-Q2	10
29	Worked answer M02-Q2	10
30	Practice M02-Q3	10
31	Worked answer M02-Q3	11
32	Reading explanation	11
33	Strings and numerical search	12
34	Reading explanation	12
35	Reading explanation	12
36	Reading explanation	13
37–38	Reading explanation	14
39–40	Reading explanation	14
41	Reading explanation	15
42	Reading explanation	16
43–44	Reading explanation	14

Source PDF pages	Reading-note destination	Notes page
45	Practice M03-Q1	16
46	Worked answer M03-Q1	17
47	Practice M03-Q2	16
48	Worked answer M03-Q2	17
49	Practice M03-Q3	17
50	Worked answer M03-Q3	17
51	Functions and program design	18
52	Reading explanation	18
53	Reading explanation	18
54	Reading explanation	19
55	Reading explanation	19
56–57	Reading explanation	20
58	Practice M04-Q1	21
59	Worked answer M04-Q1	22
60	Practice M04-Q2	21
61	Worked answer M04-Q2	22
62	Practice M04-Q3	22
63	Worked answer M04-Q3	23
64	Lists, tuples, and data integrity	24
65	Reading explanation	24
66	Reading explanation	24
67	Reading explanation	25
68	Reading explanation	25
69	Reading explanation	26
70	Reading explanation	26
71	Reading explanation	27
72	Reading explanation	25
73	Practice M05-Q1	27
74	Worked answer M05-Q1	28
75	Practice M05-Q2	28
76	Worked answer M05-Q2	29
77	Practice M05-Q3	28
78	Worked answer M05-Q3	29
79	Recursion and dictionaries	30
80	Reading explanation	30
81	Reading explanation	30
82–83	Reading explanation	31
84	Reading explanation	32
85	Reading explanation	32
86	Reading explanation	33
87	Reading explanation	31
88	Practice M06-Q1	33
89	Worked answer M06-Q1	34
90	Practice M06-Q2	34
91	Worked answer M06-Q2	35
92	Practice M06-Q3	34
93	Worked answer M06-Q3	35
94	Testing, debugging, and failure	36
95	Reading explanation	36
96	Reading explanation	36

Source PDF pages	Reading-note destination	Notes page
97	Reading explanation	37
98	Reading explanation	37
99	Reading explanation	37
100	Reading explanation	37
101	Reading explanation	38
102	Reading explanation	39
103	Practice M07-Q1	39
104	Worked answer M07-Q1	40
105	Practice M07-Q2	39
106	Worked answer M07-Q2	40
107	Practice M07-Q3	40
108	Worked answer M07-Q3	40
109	Classes, instances, and methods	42
110	Reading explanation	42
111	Reading explanation	42
112–113	Reading explanation	43
114	Reading explanation	44
115	Practice M08-Q1	44
116	Worked answer M08-Q1	45
117	Practice M08-Q2	45
118	Worked answer M08-Q2	45
119	Reading explanation	45
120	Interfaces, shared state, and inheritance	46
121	Reading explanation	46
122	Reading explanation	47
123	Reading explanation	48
124	Reading explanation	48
125–127	Reading explanation	49
128	Practice M09-Q1	50
129	Worked answer M09-Q1	51
130	Practice M09-Q2	50
131	Worked answer M09-Q2	51
132	Practice M09-Q3	50
133	Worked answer M09-Q3	51
134	Efficiency: counts and growth	52
135	Reading explanation	52
136	Reading explanation	52
137	Reading explanation	53
138	Reading explanation	53
139	Reading explanation	53
140–141	Reading explanation	54
142	Reading explanation	54
143	Reading explanation	54
144	Practice M10-Q1	55
145	Worked answer M10-Q1	56
146	Practice M10-Q2	55
147	Worked answer M10-Q2	56
148	Hidden work, recursion, and memory	57
149	Reading explanation	57
150	Reading explanation	57

Source PDF pages	Reading-note destination	Notes page
151	Reading explanation	58
152–153	Reading explanation	58
154	Reading explanation	59
155	Reading explanation	60
156	Reading explanation	61
157	Practice M11-Q1	62
158	Worked answer M11-Q1	62
159	Practice M11-Q2	62
160	Worked answer M11-Q2	63
161	Practice M11-Q3	62
162	Worked answer M11-Q3	63
163	Searching and sorting	64
164	Reading explanation	64
165	Reading explanation	64
166	Reading explanation	65
167	Reading explanation	65
168	Reading explanation	66
169	Reading explanation	66
170–171	Reading explanation	67
172	Reading explanation	67
173–174	Reading explanation	68
175	Reading explanation	69
176	Practice M12-Q1	69
177	Worked answer M12-Q1	70
178	Practice M12-Q2	69
179	Worked answer M12-Q2	70
180	Practice M12-Q3	70
181	Worked answer M12-Q3	70
182	Programming beyond Python	72
183	Reading explanation	72
184	Reading explanation	72
185	Reading explanation	72
186	Reading explanation	73
187	Reading explanation	74
188	Reading explanation	74
189	Reading explanation	75
190	Reading explanation	76
191	Practice M13-Q1	76
192	Worked answer M13-Q1	77
193	Practice M13-Q2	76
194	Worked answer M13-Q2	77
195	Practice M13-Q3	77
196	Worked answer M13-Q3	77
197	Integrated engineering review	79
198	Reading explanation	79
199	Reading explanation	79
200	Reading explanation	80
201–202	Reading explanation	80
203–204	Reading explanation	81
205–206	Reading explanation	82

Source PDF pages	Reading-note destination	Notes page
207	Reading explanation	83
208	Reading explanation	84
209	Reading explanation	84
210	Practice INT-Q1	84
211	Worked answer INT-Q1	85
212	Practice INT-Q2	85
213	Worked answer INT-Q2	85
214	Mixed-topic practice	86
215	Practice X01	86
216	Worked answer X01	88
217	Practice X02	86
218	Worked answer X02	88
219	Practice X03	87
220	Worked answer X03	89
221	Practice X04	87
222	Worked answer X04	89
223	Practice X05	87
224	Worked answer X05	89
225	Practice X06	87
226	Worked answer X06	89
227	Practice X07	87
228	Worked answer X07	89
229	Practice X08	88
230	Worked answer X08	89
231	Practice X09	88
232	Worked answer X09	90
233	Practice X10	88
234	Worked answer X10	90
235	Final checks and reference	91
236	Reading explanation	91
237	Reading explanation	91
238–240	Reading explanation	92